

Road Condition Detection

B-Tech -AI/DS, E&TC, Semester-VI

Prepared at



Bhaskaracharya National Institute for Space Applications & Geo-informatics
Ministry of Electronics and Information Technology, Govt. of India.
Gandhinagar

Prepared By

Omana Arya

ID No. 26BISAG0495

Anshika Tiwari

ID No. 26BISAG0496

Guided By:

Dr. Shweta Saharan

Department of AI&DS

GSV, Vadodara.

External Guide:

Dr. Manoj Pandya

Additional Director,

BISAG-N, Gandhinagar

SUBMITTED TO

Gati Shakti Vishwavidyalaya



Gati Shakti Vishwavidyalaya - Vadodara

**Bhaskaracharya National Institute for Space Applications
and Geo-informatics**

MeitY, Government of India

**प्रमाणपत्र
CERTIFICATE**

दिनांक/Date: _____

यह प्रमाणित किया जाता है कि गति शक्ति विश्वविद्यालय, वडोदरा की छठे सेमेस्टर की बी.टेक.-एआई एंड डीएस की छात्रा सुश्री ओमना आर्या तथा छठे सेमेस्टर की बी.टेक.-ईसीई की छात्रा सुश्री अंशिका तिवारी ने बायसेग-एन में अंतिम सेमेस्टर की इंटरशिप सफलतापूर्वक पूर्ण की है। उन्होंने 18 मई, 2026 से 13 जुलाई, 2026 तक रोड कंडीशन डिटेक्शन पर काम किया है।

जहाँ तक हमें सूचना है, यह उनके द्वारा किया गया एक मौलिक और प्रामाणिक कार्य है। संस्थान में अपने कार्यकाल के दौरान, उन्होंने आत्म-प्रेरणा और कर्तव्यनिष्ठा का परिचय दिया। हम उनके उज्ज्वल भविष्य की कामना करते हैं।

This is to certify Ms. Omana Arya and Ms. Anshika Tiwari, students of 6th Semester B.Tech-AI/DS and B.Tech-E&TC respectively from Gati Shakti Vishwavidyalaya, Vadodara, have satisfactorily completed the summer internship at BISAG-N. They have worked on Road Condition Detection, from May 18th, 2026 to July 13th, 2026.

To the best of our information this is an original and bonafide work done by them. During their tenure at the institute, they demonstrated self-motivation and conscientiousness. We wish them a successful future ahead.

मनोज पंड्या
Manoj Pandya
अपर निदेशक
Additional Director,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

पुनीत लालवानी
Punit Lalwani
अपर निदेशक सह सीआईएसओ
Additional Director cum CISO,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

GATI SHAKTI VISHWAVIDYALAYA

Certificate of Completion

This is to certify that the work contained in the project titled "**Road Condition detection**" has been successfully completed by the following students under our supervision during a **8-week** summer internship. During the internship period, the students were found to be hard-working, punctual, and disciplined, and they met all the criteria satisfactorily.

Student Name	Roll Number	Signature
Omana Arya	23AI036	
Anshika Tiwari	23EC012	

Guide

Dr. Shweta Saharan

Assistant Professor

Program Coordinator

Dr. Vipul Kumar Mishra

Associate Professor



Supervisors / Industry Mentors:

Dr. Manoj Pandya

Signature: _____

About BISAG-N



Modern day planning for inclusive development and growth calls for transparent, efficient, effective, responsive and low-cost decision-making systems involving multi-disciplinary information such that it not only encourages people's participation, ensuring equitable development but also takes into account the sustainability of natural resources.

The applications of space technology and Geo-informatics have contributed significantly towards the socio-economic development. Taking cognizance of the need of geo-spatial information for developmental planning and management of resources, the department of Ministry of Electronics and Information Technology, Government of India, established "Bhaskaracharya National Institute for Space Applications and Geo-informatics" (BISAG-N).

BISAG-N is ISO 9001, ISO 27001, ISO 19115, ISO 19157 and CMMI: L5 certified national institute. BISAG-N, which was initially set up to carry out space technology applications, has evolved into a center of excellence, where research and innovations are combined with the requirements of users and thus acts as a value-added service provider, a technology developer and as a facilitator for providing direct benefits of space technologies to the grass root level functions/functionaries.



BISAG- N Journey

The journey of Institute started as a state government organization, named as **Remote Sensing and Communication Centre (RESECO)** operating under a limited mandate from 1997 to 2003, focusing on utilization of space technologies such as satellite communication, remote sensing and geospatial applications for supporting various developmental initiatives exclusive for the State of Gujarat. Opinions of various stakeholders at grass root levels was obtained to ensure alignment of Institute's work with the development and welfare need of the society. Numerous sessions of different strata of society were organized including Member of Legislative Assembly (MLAs), village sarpanch, farmers, various skill workers, women and students to obtain insights into what are the needs with simplicity of solutions, and processes of their groups and how technology can fulfil those needs and alleviate any constraints for improving the quality of life.

The basic works of maps generation as per the needs of various state government departments laid the groundwork for the institute's advancement. In December 2003, the name of the institute changed as **BISAG (Bhaskaracharya Institute for Space Applications and Geo-informatics)** with an enhanced mandate to promote satellite communication-based education and skill development facility thereby ensuring increased and more actively involvement in governance processes. The name of the institute was changed to honor the celebrated 12th-century mathematician and astronomer, Bhaskaracharya (1114-1185 AD) who provided in-depth insights about various astronomical and mathematical concepts, formulae and calculations.

The institute's services were extensively utilized by various departments of Gujarat Government under the leadership and patronage of the then Honorable Chief Minister Shri Narendrabhai Modi. Numerous times the assembly meetings were held in BISAG-N premises to amalgamate governance and technology for the well-being of the state. The state leadership expressed to the ministers and bureaucrats on multiple occasions the importance of visualization of development plans and schemes on maps and thereby harnessing the capabilities of BISAG as a **"go-to place"** for various technological and geo-informatics solutions required by concerned ministries and departments for the benefit of citizens and development of the state.

On one of such occasions in 2004, Honorable Chief Minister Shri Narendra Modi said, "Friends, I can clearly visualize that within a span of five years, all streams of government administration shall converge into BISAG. This institute shall be a huge treasure of information, if any decisions are to be taken or any department require any information or need any policy change; the **'Source of Information'** shall be BISAG. There for earlier we can transform BISAG into a **'Source of Information'** and position it as back-bone of all governance and administration activities, and the sooner the benefits of science and technology shall be available to the society."



Since then, inspired by above and gaining insights while working collaboratively for various development initiatives of central and state government department, the Institute is focused on following vision:

“To empower and serve the nation through development of space and geo-spatial technologies, its applications, solutions and services in accordance to governance principles for socio-economic welfare of the society.”

The Institute develops geo-spatial technologies, databases, software and processes along with skilled resources to provide visualization, analysis and decision support system to various state government departments. Also, the Satellite Communication infrastructure was enhanced to add more channels and developed in house expertise to install, operate and maintain various electronic instruments and equipment. The institute constantly ensured adoption of national and international standards as well as best practices to develop and reinforce the solutions delivery process. With the improved confidence and trust of the state government to utilize geo-spatial and satellite communication services for developmental, social welfare and disaster management areas, the institute was supported by the Government of Gujarat to construct dedicated buildings for ensuring services are provided at even larger scale. This expansion broadened the institute's scope to integrate advanced techniques such as managing large databases, mathematical modelling, enabling prediction and simplified architecture for the solution aligning with the government priorities for improving efficiency, transparency and pace of development

Along with providing the e-governance support to Gujarat State, the institute also started supporting various other states and central government ministries as and when required for map and decision support system development services. Recognizing this potential, the institute's mandate was further enhanced to cater to the geo-spatial and satellite communication-based e-governance services to all central government ministries as well as various states and union territories as a **National** institute. The name of the institute was then changed as Bhaskaracharya **National** Institute for Space Applications and Geo-informatics (**BISAG-N**), registered as an Autonomous Scientific Society, as per the Societies Registration Act, 1860, under the Ministry of Electronics and Information Technology (MeitY), Government of India (GoI). BISAG-N; enhanced mandate includes to undertake technology development & management; research & development; facilitate national and international cooperation and capacity building; support technology transfer and entrepreneurship development in area of geo-spatial technology.

Today, BISAG-N is collaboratively working with 50+ central government ministries and most of the states and union territories for supporting various developmental programs and citizen centric services. BISAG-N provides the technological support to these stakeholders from its Gandhinagar and New Delhi facilities. These facilities have adequate skilled personnel and state-of-the-art infrastructure for providing geo-spatial solutions and software applications including support of satellite communication-based education and training. Thus, BISAG-N is playing pivotal role as a one stop center for providing geo-spatial and space technologies for various flagship programme being undertaken for the development of nation in line with geo-spatial education, logistics and various other policies and program of central and state governments.



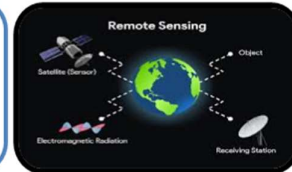


Satellite Communication:

For promotion and facilitation of the use of broadcasting facility for distant interactive training, education, skill development, health, agriculture and extension

Remote Sensing:

For inventory, mapping, developmental planning and monitoring of natural and built resources

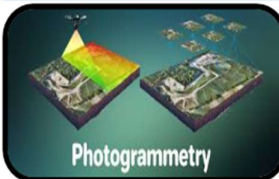


Geographic Information System:

For conceptualization, creation and organization of multi-purpose common digital databases for development of sectoral/integrated decision systems.

Global Navigation Satellite System:

For location based services, engineering applications and research

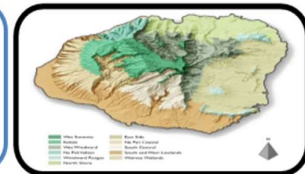


Photogrammetry:

For creation of maps, Digital Elevation Model (DEM), terrain characteristic maps for resource planning and management

Cartography:

For Thematic Mapping and generation of value added maps



Software Applications Development:

For wider usage in geo-spatial applications, Decision Support Systems (DSS) and ERP solutions.

Innovation, Research & Development:

Quantum Computing, Digital Twins, Cyber Security, Block Chain, Big Data, AI/ML

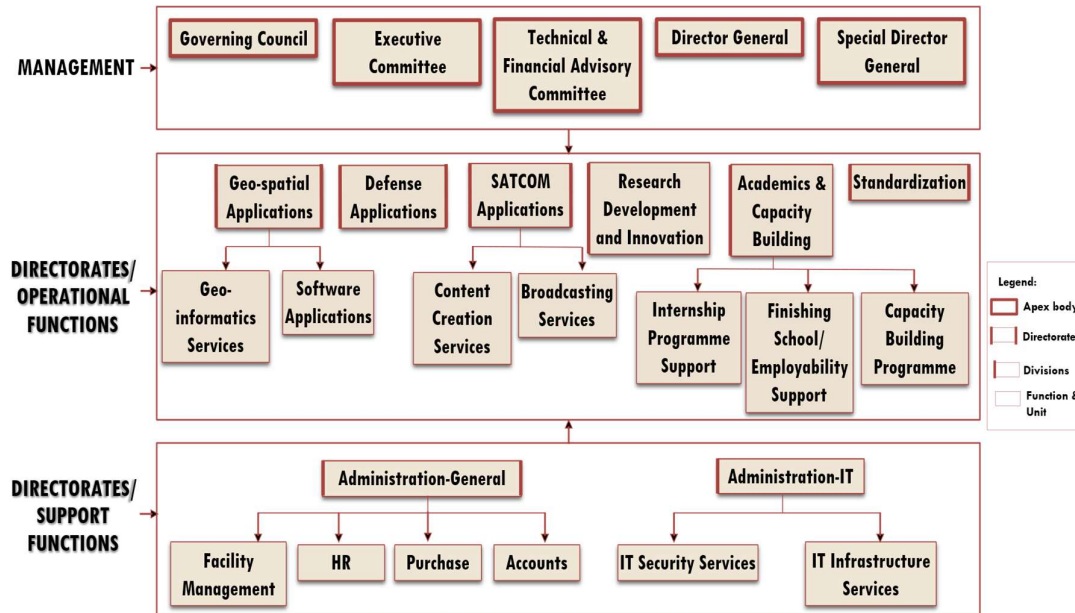


Education and Training:

For providing Education, Training & Technology knowledge transfer to large number of students, end users and collaborators for promoting Research & Development

Organizational Setup

BISAG-N main domain areas include Satellite Communication, Geo-spatial Solutions and Technology Development, Geo-informatics Applications Development, Standardization, Academics & National/ International Co-operation and Administration.



BISAG-N: Functions and Divisions

SATCOM Applications

The SATCOM Directorate provides services of distance education, capacity building, training, extension, awareness and empowerment using the state-of-the-art facility comprises of an uplink earth station, control room, studios and a satellite communication-based network. The services are primarily utilized by Ministry of Education and various other central and states ministries. Through Ministry of External Affairs (MoEA), BISAG-N is now poised to provide SATCOM based education to neighbouring countries also.

SATCOM operations are inspired by the wisdom of Hon'ble Prime Minister of India as elicited on numerous occasions. For example, in 2004 as Hon'ble Chief Minister of Gujarat, Shri Modi explained the importance of learning through the ancient way of visualization using virtual media with these words:

“The way Eklavya studied by making a statue, why we can't study with the help of a T.V.? Yes, we can. Was any cinema song ever taught to us in any class? Was it ever taught by any teacher? We learnt song from T.V., isn't it? If we have learnt entire cinema song, learnt every word, learnt scale, learnt rhythm, it means we can learn through T.V. also.”



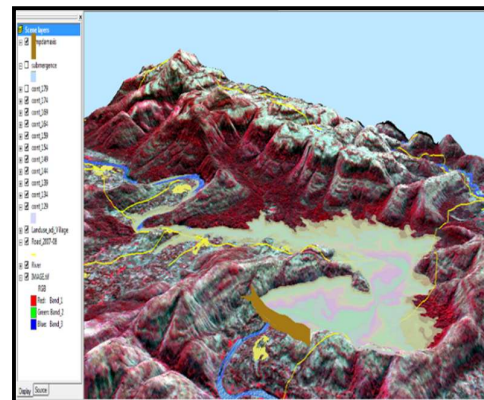
There are twelve state-of-the-art studios for live recording of content supported by dedicated editing labs, subtitling, translation, animation and various other aspects. The institute has set up ten studios at Gandhinagar facility and another two studios at New Delhi. Infrastructure for transmission and Ku band uplink teleports (11m/9.3m) of more than 350 channels is installed, integrated and maintained by inhouse team. BISAG-N support various initiatives of Ministry of Education, Government of India, as well as other central and state government ministries such as PM e-Vidya, Swayam Prabha etc. as detailed further.



Geo-spatial Applications Directorate

BISAG-N's Geo-Spatial Applications Directorate provides specialized services and solutions to implement Geographic Information Systems (GIS) for providing impetus to planning and developmental activities at grass root level. The objective of GA directorate is to provide analysis and decision support services to various stakeholders through scientifically organized, comprehensive, multi-purpose, compatible and large scale geo-spatial databases.

Geo-spatial Applications Directorate creates and organizes large-scale and multi-purpose geospatial datasets in collaboration with central and state government departments and agencies. A well-qualified team of GIS technicians and engineers ensure that the datasets are compatible, standardized and authenticated for use in analysis, planning and decision-support functions. Goals and roles of converging organizations are taken into consideration while geo-spatial database creation and software development activities



Software Development Division

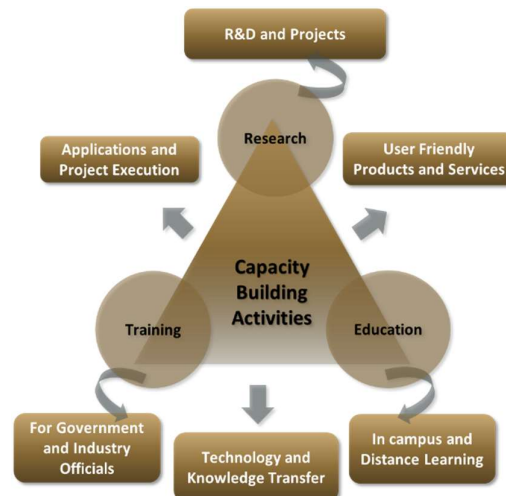
The geo-spatial solutions are packaged and provided as software products for ease of retrieval of spatial and feature attributes data, enabling wide range of analysis, prediction and decision support services. Software technologies like Java, Python, JavaScript, Spring Boot, React JS, Node JS, Microservices, XML, GeoJSON, GeoServer, QGIS, Android and iOS frameworks are used to develop user-friendly and reliable solutions.



Academy of Geo-informatics

BISAG-N, being a national institute for geo-spatial and emerging technology-based applications, has a strong focus on research, development, capacity building and employability support for under-graduate, post-graduate and doctoral students. The institute's infrastructure and supportive work environment provide an ideal platform for students pursuing thesis work in a variety of technologies and domains.

BISAG-N is equipped with cutting-edge infrastructure to support research across multiple disciplines. Dedicated laboratories for research on the application of artificial intelligence, machine learning, GIS (Geographic Information Systems), remote sensing, quantum computing, IoT (Internet of Things) and other emerging technologies are established high-performance computing facilities for intensive data processing tasks are available in the institute. Open-source platforms and in-house developed software are available for projects, research and innovation activities. More than 300 students are supported every year for learning practical based aspects of geo-spatial and emerging technology applications while completing their projects.



BISAG-N Finishing School Programme (FSP) supports wards of special groups of society such as scheduled tribe, scheduled caste, non-gazetted security, armed forces citizens and martyr etc, for development of technical and soft skills to improve their employability including support for entrepreneurship and employment opportunities.

The school provides basic and specialized training to help participants develop capacities as required by the for related industry/domain. The participants undergo thorough counselling sessions as well as mock/actual interviews for improving employability in relevant industries as per the interest of the candidates. Along with the skills training the participants are also allowed to observe engaged in ongoing projects of organization. BISAG-N academic activities also provide capacity building support to government officials for effectively using geo-spatial solutions and decision support systems for performing planning and monitoring for various government schemes and initiatives.

Industrial visits of school students are organized to provide them exposure of space applications and geo-informatics thereby supporting the development of scientific temperament at early stage. More than 5000 students visit the premises every year to get exposure of the activities being carried out at BISAG-N.



Standardization Directorate

BISAG-N has a dedicated directorate for ensuring continuously focus on innovation, standardization and institutionalization of solutions & service. The Standardization Directorate focuses on implementation of various national and international standards related to program solutions and data management. The directorate develops internal SOPs and policies for compliance to these standards and provides training for effective implementation. The directorate also liaison with the various departments and external agencies during internal and external audits. In addition, the directorate acts as member in standards committees of Bureau of Indian Standards (BIS) and other standard developing bodies for development of standards.

Goals of Standardization Directorate include:

- To proactively implement product and process improvements initiatives using various national & international standards and best practices. This includes ensuring quality and reliability of geo-spatial data, completeness, updation of metadata, functional performance and security reviews of applications, capacity building support to end-users for adoption of solutions and information security-oriented culture development
- To develop a culture of efficiently working as per standards while fostering innovation
- To perform reviews, gap analysis, training, documentation and standards implementation support activities

Administration Directorate

Administrative Directorate provides strategic and management support to various activities of BISAG-N. The directorate ensures infrastructure and facilities management in addition to providing services such as coordination during visits of various delegations, logistic support to BISAG-N staff during travel, accounting and finance, procurement, training, safety and security, legal compliance, maintenance, housekeeping, horticulture, construction and extension projects, campus monitoring, canteen management as well as celebrations and welfare activities. The directorate also ensure recruitment and human resource development as per the needs of various projects and directorates while upholding principles of equality, fairness and diversity in work force.

BISAG-N has adequate human resources, well trained for behavioral, functional and domain competencies, with clear roles and responsibilities in alignment with service delivery expectations. The staff well-being and motivation initiatives are ensured for maintaining a culture of shouldering responsibilities proactively and delivering high-performance utilizing multi-disciplinary knowledge. Administration Directorate ensure strategic and compliance support to operations of the institute by managing human resources, facilities, expansion projects, finance and governance aspects. The directorate also coordinates with Council stakeholders and members of Governing and Executive committee as well as other high-level committees.

Defence Applications Directorate

This directorate develops space, geo-spatial, AI and other emerging technologies-based applications under the guidance of various agencies of the Ministry of Defence, Government of India. All the technologies used are developed inhouse of customized open source. Considering the strategic significance of these projects, dedicated team and premises are allocated for ensuring information security. Most of the critical activities and data management are performed directly at various defence establishments.

Besides the projects of various infrastructure and social welfare ministries and departments of central and state government, BISAG-N has a dedicated directorate for supporting defence and security departments for providing data products, decision support systems and services involving geo-spatial and various other emerging technologies. The projects are executed in high security dedicated laboratory for various units of army, navy and air-force for supporting strategic and operational aspects of the operations.

Through its innovative geospatial solutions and technological expertise, BISAG-N is playing a crucial role in enhancing the nation's defence capabilities and ensuring long-term strategic autonomy. Due to nature of high secrecy of these strategic projects, they are not included in details in this compilation.

Research and Development

BISAG-N's research and development group supports all other directorates for providing solutions related to new definitions and technology. The group develops innovative applications, features, tools and services utilizing technologies such as remote sensing, photogrammetry, artificial intelligence, machine learning, internet of things, computer vision, big data and quantum computing, which are later integrated in live projects.

BISAG-N has made significant progress for the development of Artificial Intelligence/ Machine Learning (AI/ML) applications. Algorithms and models have been developed which can be customized and used with training, validation and live data upon their data for accurate predictions and smarter results.

The applications developed by the R&D team include support for the pluggable APIs which can use the machine learning with minimal code reconfiguration. Various GIS applications are developed at BISAG-N. Utilizing spatial and non-spatial processing of Big Data and Web GIS architecture These applications support multi-terabyte datasets and large number of consumers. Big Data applications for visualization developed by the institute has enabled several projects to explore and analyse multiple large archives. Through continuous research and development, BISAG-N's R&D capabilities are continuously enhanced. This allows to enhance users' satisfaction, growth and success stories.

Cyber Security Cell

Cyber Cell reviews and update the threat landscape for the BISAG-N's critical infrastructure on a regular basis in line with various vulnerabilities detected and published by reputed repositories. The cell verifies the applications using Vulnerability Assessment & Penetration Testing (VAPT) ensuring the compliance to Guidelines for Indian Government Websites (GIGW), Open Web Application Security Project (OWASP), SANS 25 etc. The cell also ensures implementation of good information security practices in line with ISO 27001 such as serious logs review, vulnerability analysis, exception reporting, maintenance of firewall and other network monitoring tools.

CANDIDATE'S DECLARATION

We declare that the 6th semester internship project report entitled “**Road Condition Detection**” is our own work conducted under the supervision of the external guide **Dr. Manoj Pandya** from BISAG-N (Bhaskaracharya National Institute for Space Applications & Geo-informatics). We further declare that to the best of our knowledge, the report for this project does not contain any part of the work which has been submitted previously for such project either in this or any other institutions without proper citation.

Candidate 1's Signature

Omana Arya

Student ID: 26BISAG0495

Candidate 2's Signature

Anshika Tiwari

Student ID: 26BISAG0496

Submitted To:

Gati Shakti Vishwavidyalaya - Vadodara,

ACKNOWLEDGMENT

We are grateful to **Shri T.P. Singh**, Director General (BISAG-N), for giving us this opportunity to work the guidance of renowned people of the field of GIS-Based Portal, also providing us with the required resources in the company.

We would like to express our endless thanks to our external guide, **Dr. Manoj Pandya**, our coordinator **Dr. Abdul Jhummarwala**, and to the Training Cell, **Mr. Punit Lalwani & Mr. Siddharth Patel** at Bhaskaracharya National Institute of Space Application and Geo-informatics, for their sincere and dedicated guidance throughout the project development.

Also, our hearty gratitude to our Head of Department, **Dr. Vipul Kumar Mishra** and **Dr. Shweta Saharan** for giving us encouragement and technical support on the project.

Omana Arya.

Student ID: 26BISAG0495

Anshika Tiwari.

Student ID: 26BISAG0496

ABSTRACT

Roads are really important for getting people and things from one place to another and on time.. Checking the condition of roads by hand takes a lot of time and work and people can make mistakes. This project is about a system that uses intelligence to automatically find and analyze problems with the road surface and things on the side of the road like trees and signs using computer vision and deep learning.

The system uses a tool called YOLOv11 to find classify and separate different kinds of road damage like potholes, cracks and broken pavement as well as things on the side of the road like trees, signs and guard rails. We made our own set of pictures to teach the system how to do this. When the system looks at videos of roads it can find problems. Highlight them right away. To make the system more useful we added a way to track and count things so each thing gets an identity and is not counted twice. The system can also figure out how bad the road condition is based on what it finds and it can make reports, with statistics. We also made a dashboard where people can see the results, counts and summaries in a way that's easy to understand.

This system is meant to help with checking roads watching over infrastructure and planning maintenance by giving accurate and efficient analysis of road conditions. The project shows how artificial intelligence and computer vision can be used in transportation and infrastructure management systems. Road condition detection is what this system does. It uses Road Condition Detection to make roads safer and more efficient.

CONTENTS

Chapter No.	Description	Page No.
-	List of Figures	I
-	List of Tables	II
-	List of Abbreviations	IV
-	List of Definitions	V
-	List of Screenshots	VII
1	Introduction	1
1.1	Project Details	1
1.2	Purpose	1
1.3	Scope	2
1.4	Objectives	3
1.5	Technology Stack Used	3
1.6	Literature Review	5
2	Dataset Preparation	7
2.1	Data Collection	7
2.2	Data Sources	7
2.3	Dataset Statistics	8
2.4	Class Definition and Label Schema	9
2.5	Annotation Process	11
2.5.1	Annotation Tool — Roboflow	11
2.5.2	Role of SAM in Annotation	12
2.6	Dataset Splitting (Train / Val / Test)	13
2.7	Dataset Challenges and Limitations	14
3	System Architecture	15
3.1	Overall Pipeline Overview	15
3.2	Input Module	16

Chapter No.	Description	Page No.
3.3	Preprocessing	16
3.4	Detection and Segmentation Module	17
3.5	Measurement Module	18
3.5.1	Area Estimation- GSD Method	18
3.6	Output Module	19
4	Training	20
4.1	Training Environment	20
4.2	Pretrained Weights and Transfer Learning	23
4.3	Model Progression	24
4.3.1	Road Damage Model	24
4.3.2	YOLOv26l-seg	25
4.3.3	Road Surface Mask Model	26
4.4	Hyperparameter Configuration	26
4.5	Data Augmentation Strategy	28
5	Implementation	30
5.1	Development Environment	30
5.2	Inference Pipeline	31
5.3	Mask Alignment and Rendering	33
5.4	Multi-Object Tracking Using ByteTrack	36
5.6	Road Surface Masking	38
6	Dashboard	40
6.1	Dashboard Features	40
6.2	Dashboard Screenshots	41
7	Results and Evaluation	43
7.1	Evaluation Metrics	43
7.2	Results: YOLOv11s-seg	43
7.3	Results: YOLOv11m-seg	43
7.4	Results: YOLOv11l-seg	44

Chapter No.	Description	Page No.
7.5	Results: YOLOv26l-seg	44
7.6	Results: YOLOv11l-seg Road Mask Model	45
7.7	Confusion Matrix	46
7.8	PR Curve Analysis	46
7.9	Sample Detection Outputs	47
7.10	Area and Length Measurement Results	48
9	Conclusion and Future Work	51
9.1	Summary of Work Done	51
9.2	Key Findings	52
9.3	Future Enhancements	53
9.4	Real-World Deployment Considerations	55
10	References	57
11	Appendix	62

List of Figures

Figure No.	Description	Page No.
1.1	Project pipeline architecture	4
2.1	Image from dataset	7
2.2	Classes used in dataset	10–11
4.1	Training curves (YOLOv11-seg)	25
4.2	Image with road mask	26
5.1	Block Diagram of the Proposed Video Processing Framework	32

List of Tables

Table No.	Description	Page No.
1.1	Project details	1
1.2	Technology Tools used	3
2.1	Class-wise Annotation Count (Training Set)	8
2.2	Dataset Split Distribution	13
3.1	Input Module Parameters	16
3.2	Default Camera Parameters Used for GSD Calculation	19
4.1	GPU Platform Comparison	21
4.2	Per-Model Training Environment Summary	21
4.3	Ultralytics Hub Training Environment	22
4.4	Kaggle Training Environment	23
4.5	Parameters of Road Mask Model	26
4.6	Hyperparameter Configuration: YOLOv11s and YOLOv11m	27
4.7	Hyperparameter Configuration: YOLOv11l and YOLOv26l	27
4.8	Geometric Augmentation Parameters	28
4.9	Photometric Augmentation Parameters	29
5.1	Development Environment Specifications	30
5.2	Mask Rendering Parameters	35
5.3	Class Colour Assignments (BGR)	35
5.4	ByteTrack Configuration	37
7.1	Evaluation Metric Summary	43
7.2	YOLOv11l-seg Overall Results (Ultralytics Hub)	44
7.3	YOLOv26l-seg Overall Results (Ultralytics Hub)	45
7.4	YOLOv11l-seg Road Mask Model Results	45
7.5	Area Measurement Results from Test	48

7.6	Crack Length Measurement Results from Test Video	48
7.7	Unique Object Count Results from Test Video	49

List of Abbreviations

Abbreviation	Full Form
ML	Machine Learning
CNN	Convolutional Neural Network
R-CNN	Region-based Convolutional Neural Network
SSD	Single Shot (multibox) Detector
YOLO	You Only Look Once
SAM	Segment Anything Model
GSD	Ground Sampling Distance
IoU	Intersection over Union
mAP	mean Average Precision
NMS	Non-Maximum Suppression
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory
API	Application Programming Interface
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
FPS	Frames Per Second
GPS	Global Positioning System
GIS	Geographic Information System
PCI	Pavement Condition Index
RDDC	Road Damage Detection Challenge
KPI	Key Performance Indicator
BGR	Blue-Green-Red (colour channel order)
HSV	Hue-Saturation-Value
SGD	Stochastic Gradient Descent
COCO	Common Objects in Context (dataset)
CRF	Constant Rate Factor

List of Definitions

Term	Definition
AdamW Optimizer	An optimization algorithm that updates model weights during training using adaptive learning rates and decoupled weight decay for improved generalization.
Batch Size	The number of training samples processed together before the model parameters are updated.
Bilinear Interpolation	An image resizing technique that estimates new pixel values using the weighted average of the four nearest neighbouring pixels.
ByteTrack	A multi-object tracking algorithm that assigns persistent identities to detected objects across consecutive video frames, enabling accurate unique object counting.
Class Imbalance	A condition where some classes contain significantly more training samples than others, potentially affecting model performance.
COCO Dataset	A large-scale public dataset containing images of common objects, widely used for pre-training object detection and segmentation models.
Confidence Score	The probability assigned by the model indicating how confident it is that a detected object belongs to a particular class.
Confusion Matrix	A matrix that summarizes the classification performance of a model by comparing predicted classes against the ground-truth classes.
Copy-Paste Augmentation	A data augmentation technique that increases dataset diversity by copying segmented objects from one image and pasting them into another.
Cosine Learning Rate Schedule	A learning rate scheduling technique that gradually reduces the learning rate following a cosine function during training.
Data Augmentation	The process of artificially increasing dataset diversity by applying transformations such as rotation, flipping, scaling, and colour adjustments.
Epoch	One complete iteration through the entire training dataset during model training.
FFmpeg	An open-source multimedia framework used to convert the processed output video into the H.264 format for browser compatibility.
Ground Sampling Distance (GSD)	The real-world distance represented by a single pixel in an image, used to estimate the physical area of detected road damage.

H.264	A widely used video compression standard employed for the final output video generated by the system.
Instance Segmentation	A computer vision task that detects individual object instances and generates a pixel-level segmentation mask for each detected object.
Intersection over Union (IoU)	A metric used to measure the overlap between a predicted segmentation mask and the corresponding ground-truth mask.
Kaggle	A cloud-based platform providing GPU-enabled environments for machine learning model development and experimentation.
Label Smoothing	A regularization technique that reduces model overconfidence by replacing hard class labels with softened probability distributions during training.
Learning Rate	A hyperparameter controlling the magnitude of weight updates during the optimization process.
Letterboxing	An image preprocessing technique that resizes images while preserving their aspect ratio by adding padding where necessary.
Mean Average Precision (mAP)	A standard evaluation metric used to assess the overall detection and segmentation accuracy of an object detection model across all classes.
mAP50	Mean Average Precision computed at an Intersection over Union threshold of 0.50.
mAP50–95	Mean Average Precision averaged over Intersection over Union thresholds ranging from 0.50 to 0.95, providing a stricter evaluation of model performance.
MixUp	A data augmentation technique that combines two training images and their labels to create a new synthetic training sample.
Non-Maximum Suppression (NMS)	A post-processing technique that removes multiple overlapping detections of the same object while retaining the highest-confidence prediction.
OpenCV	An open-source computer vision library used for video processing, frame extraction, image manipulation, and visualization.
Overfitting	A condition where a model performs well on training data but poorly on unseen data because it has learned training-specific patterns rather than general features.
Photogrammetry	The science of obtaining real-world measurements from images, used in this project to estimate the area of detected road damage.

Precision	The proportion of correctly predicted positive detections among all predicted positive detections.
Precision–Recall Curve	A graph illustrating the relationship between precision and recall at different confidence thresholds for evaluating detection performance.
PyTorch	An open-source deep learning framework used as the backend for training and deploying the YOLO models.
Recall	The proportion of actual objects correctly detected by the model.
Roboflow	A cloud-based platform used for dataset annotation, dataset management, preprocessing, and export in YOLO segmentation format.
Segment Anything Model (SAM)	A foundation segmentation model developed by Meta AI that automatically generates accurate segmentation masks from simple user prompts.
Streamlit	An open-source Python framework used to develop the interactive dashboard for video upload, inference, and visualization of results.
Transfer Learning	A machine learning technique in which a pre-trained model is fine-tuned on a new dataset to improve training efficiency and performance.
Ultralytics	The organization and software framework providing the implementation, training, and inference pipeline for YOLO models.
Ultralytics Hub	A cloud-based training platform provided by Ultralytics for managing datasets, training experiments, and model deployment.
YOLO (You Only Look Once)	A family of real-time object detection and instance segmentation models that perform object localization and classification in a single forward pass.
YOLO11	An advanced version of the YOLO architecture used in this project for both road damage detection experiments and road surface segmentation.
YOLO26	The latest YOLO architecture used as the final road damage detection model in this project due to its superior detection performance.

List of Screenshots

Screenshot No.	Description	Page No.
2.1	Roboflow annotation interface	12
2.2	SAM-assisted annotation in Roboflow showing auto-generated mask on tree	13
3.1	Comparison: distorted mask vs corrected mask	18
4.1	Ultralytics Hub Run Information page	22
4.2	Performance Metrics of the Trained YOLO26l-seg Model	26
5.1	Output video with mask rendering	36
6.1	Dashboard main page	41
6.2	Dashboard live detection page	41
6.3	Dashboard detection page	42
7.1	Performance Metrics of the Trained YOLOv11l-seg Model	44
7.2	Performance Metrics of Road Mask Model	45
7.3	Confusion Matrix of YOLOv26l-seg model	46
7.4	PR Curve of YOLOv26l-seg model	47
7.5	Output frame showing multiple damage classes detected simultaneously	47
7.6	Frame showing tree and guard rail detection on a road with roadside infrastructure	48
7.7	JSON file showing information summary	49
7.8	Dashboard showing information summary	50

1. Introduction

1.1 Project Details

Field	Details
Project Title	Road Condition Detection
Organization	Bhaskaracharya National Institute for Space Applications and Geo-informatics (BISAG-N), Gandhinagar, Gujarat
Internship Duration	18 May 2026 – 13 July 2026
Interns	Omana Arya and Anshika Tiwari
Degree & Branch	B.Tech – Artificial Intelligence & Data Science / Electronics and Communication Engineering
University	Gati Shakti Vishwavidyalaya
Mentor	Dr. Manoj Pandya
Guide at University	Dr. Shweta Saharan

Table 1.1- Project details

1.2 Purpose

The Road Condition Detection system helps. Check road conditions using intelligence. Engineers usually have to go out and look at roads to see what is damaged. They write down what they find, which takes a lot of time and work. It's also hard to do this for roads. This system is better because it uses videos of roads taken from a moving car to find parts. It can automatically measure damage like potholes, cracks and broken road surfaces. It can also see things like trees, guard rails and traffic signs. The system gives information like the number of parts the size of damaged areas and the length of cracks. It also gives a video that shows where the damage is. All this information is shown on a screen that we can interact with. This helps people in charge of fixing roads make decisions faster and based on information without having to go out and look at roads themselves.

We worked on this project when we were interns at **Bhaskaracharya National Institute for Space Applications and Geo-informatics** also known as **BISAG-N** in Gandhinagar. They are doing work to automate the process of checking roads in India. Road Condition Detection system is very useful, for this kind of work. The Road Condition Detection system makes it easy to check roads. It helps people fix roads quickly.

1.3 Scope

Included:

- Detection and instance segmentation of 9 road damage and roadside object classes from monocular video footage.
- Unique object counting for potholes and trees using ByteTracker
- Area estimation of potholes and ravelling in pixels and square metres using camera-based Ground Sampling Distance (GSD) formula
- Length estimation of cracks in pixels and metres.
- Road surface masking to restrict damage detection to the road area only
- A Streamlit-based dashboard for video upload, result visualisation, and JSON export
- Training pipeline on GPU resources (Kaggle & Google Colab) using YOLOv11l-seg and Yolo26l-seg.

Future Scope:

- GPS or GIS-based spatial mapping of damage locations
- Real-time on-device deployment (edge/embedded systems)
- Structural severity scoring or pavement condition index (PCI) calculation
- 3D depth estimation or stereo camera input
- Integration with government road databases

The system is designed to work on standard road survey videos recorded from a vehicle-mounted camera under normal daylight conditions on Indian urban and rural roads.

1.4 Objectives

The key objectives of this project are:

1. To build a deep learning-based segmentation model capable of detecting and classifying road damage types - pothole, crack, alligator cracking, ravelling, and rutting and roadside objects - tree, guard rail, traffic sign, and line marking from road survey video.
2. To annotate a custom dataset of road survey images collected from Indian urban and rural roads and train a YOLOv11l-seg model on it using GPU-accelerated cloud environments.
3. To implement object tracking to assign unique identities to detected objects across frames, enabling accurate cumulative counting of potholes and trees without double-counting.
4. To calculate the physical dimensions of road damage, specifically the area of potholes and ravelling in square metres, and the length of cracks in metres , using the Ground Sampling Distance (GSD) photogrammetric formula derived from camera parameters.
5. To apply a road surface mask so that damage classes are detected only within the road boundary, reducing false positives from surrounding terrain.
6. To develop an interactive Streamlit dashboard that allows users to upload a road survey video, run inference, and view annotated output, detection statistics, area/length measurements, and a per-frame detection chart with Json export.

1.5 Technology Stack

Layer	Tool / Library	Purpose
Deep Learning Model	YOLOv11l-seg & YOLO26l-seg (Ultralytics)	Instance segmentation of road damage and objects
Segmentation Assistant	SAM (Segment Anything Model)	Assisted annotation refinement
Dataset Annotation	Roboflow	Image labelling in YOLO segmentation format
Training Environment	Google Colab (T4 GPU) and Kaggle (dual T4 GPU)	Free GPU cloud training

Programming Language	Python 3.12	Core development language
Computer Vision	OpenCV	Video I/O, mask drawing, road masking
Deep Learning Framework	PyTorch	Model backbone and training
Object Tracking	ByteTracker	Unique ID assignment across frames
Mask Processing	scikit-image	Crack length measurement
Dashboard	Streamlit	Interactive web-based result visualisation
Video Encoding	FFmpeg	H.264 re-encoding for browser-compatible output
Data Handling	NumPy, Pandas	Array operations and JSON export

Table 1.2- Technology Tools used

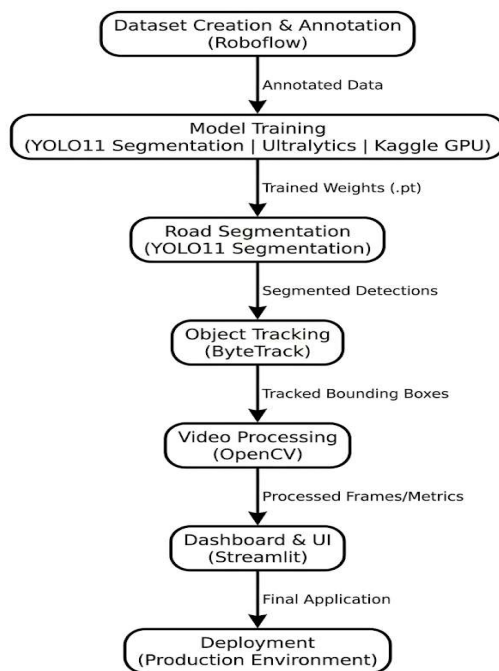


Figure 1.1- Project pipeline architecture

1.6 Literature Review

The following reviews talk about the areas of research that this project is based on.

i. Traditional Road Damage Detection

In the past people used to check the condition of roads by manually inspecting them or by using special vehicles with sensors like accelerometers and laser profilers. These methods are accurate. They are also expensive and slow. They need equipment too. Some studies like the Pavement Condition Index (PCI) came up with a way to rate how bad the road damage is, which is something this project uses when it classifies the damage.

ii. Deep Learning for Pavement Distress Detection

After 2016 people started using Convolutional Neural Networks (CNNs) to detect road damage. Zhang and others did a study in 2016 where they used CNNs to detect cracks in pavement images. It worked better than traditional methods. Later people used object detection methods like Faster R-CNN and SSD to find potholes and cracks.

iii. YOLO-Based Road Damage Detection

The YOLO models can detect things in time which makes them good for video-based road surveys. Some studies used YOLOv5 and YOLOv8 to detect road damage. They worked well. There are datasets like the Road Damage Detection Challenge (RDDC) that have become tests for this kind of thing. This project uses YOLOv11-seg, which's an extension that can segment instances so it can give a detailed mask of the damage instead of just a box around it.

iv. Instance Segmentation for Road Surfaces

Instance segmentation gives detailed information about the damage than just a box around it. It lets us analyze the area and shape of the damage. Mask R-CNN was one of the models to do instance segmentation and later models were even better. This project uses segmentation masks to calculate the area of the damage, which's more accurate than using boxes.

v. Segment Anything Model (SAM)

The Segment Anything Model (SAM) from Meta AI is a model that can segment anything. It was trained on over 1 billion masks. This project used SAM to help annotate the data so it could generate masks for the road damage, which made the annotation process easier and more accurate.

vi. Photogrammetric Area Estimation from Monocular Video

To estimate the size of things in the world from a video you need to know the geometry of the camera. The Ground Sampling Distance (GSD) formula is used to convert pixel measurements to units. This project uses the GSD formula to estimate the area of potholes and the length of cracks from video frames without needing cameras or sensors.

vii. Road Survey Video Analysis Systems

Recently people have been working on systems that can analyze road survey videos from start to finish including detection, tracking and mapping. Some systems have been developed for highway authorities, in India. They use cameras and GPS to make maps of the damage. This project focuses on the detection and measurement part of this process. It is implemented as a dashboard that can be used in the field.

2. Dataset Preparation

A good dataset is the base of any computer vision system that uses learning. The quality and variety of the training data determines how well the model works in life. This chapter talks about how the dataset was prepared for this project from collecting videos and finding sources to annotating them using Roboflow and the Segment Anything Model and finally splitting them into training and testing sets.

2.1 Data Collection

The main data for this project was collected from videos taken while driving on the road. A camera was put inside or on the vehicle. It took continuous videos of the road and the things around it as the vehicle moved on different types of roads.

The videos were taken in both cities and villages in India so they show a lot of road conditions, lighting and surroundings. The goal was not to focus on one place but to show the kinds of roads found in India including city roads with a lot of traffic and village roads, with few signs.

The videos were taken during the day. Then individual pictures were taken from the videos at times so that they could be labeled. This way the same pictures were not used over and over which made the dataset smaller without losing any information. The dataset is made up of road survey videos which're the main data for this project.



Fig 2.1- image from dataset

2.2 Data Sources

The data used for this project came from two places to make sure we had a lot of different types of road damage

Source 1. Public Datasets from Kaggle and Roboflow

We got most of our data from datasets on Kaggle and Roboflow. These datasets had pictures of roads with kinds of damage like potholes and cracks. They also had pictures of roads with things like traffic signs and trees. These pictures were taken from angles and in different conditions. This helped us train our model to recognize road damage in situations.

Source 2. BISAG-N Road Survey Videos

We also used videos from BISAG-N to make our data more realistic. These videos were taken from cameras on cars. Showed what roads are like in India. We took pictures from these videos. Labeled them by hand. These pictures showed us what roads are like in cities and in the country and what they look like in lighting. The BISAG-N data helped our model learn about the kinds of roads it would see in the world.

Using data from both of these sources made our model better at recognizing road damage. The data from Kaggle and Roboflow and the data, from BISAG-N road survey videos made our model more accurate and helpful. This is because we had a lot of kinds of road damage pictures and our model could learn from all of them.

2.3 Dataset Statistics

The final annotated dataset comprised approximately 1,300 images distributed across 9 classes. Table 2.1 presents the class-wise annotation count in the training split.

Class	Type	Annotation Count
Alligator Cracking	Road Damage	130
Crack	Road Damage	791
Guard Rails	Roadside Object	345
Line Marking	Roadside Object	661
Pothole	Road Damage	557
Ravelling	Road Damage	827

Rutting	Road Damage	58
Traffic Sign	Roadside Object	286
Tree	Roadside Object	1680
Total		5335

Table 2.1 — Class-wise Annotation Count (Training Set)

The dataset exhibits a significant class imbalance, which is characteristic of real-world road survey data. Certain damage types such as rutting and alligator cracking are naturally less frequent on roads, resulting in far fewer annotated instances compared to commonly occurring features such as trees and cracks.

2.4 Class Definition and Label Schema

We have nine classes for this project. These classes are for the kinds of damage we see on the road surface and for things we find on the side of the road. We are looking at these things together because when we survey roads we see damage and we see things like signs and bridges. These things help us figure out where we are. They are important to the people who take care of the roads.

Road Damage Classes

These classes are about what happens to the road surface. This is what we care about the most when we are trying to decide which roads to fix first.

Pothole is a word for a hole in the road. It happens when the road gets weak and water gets in and cars drive over it. Potholes are very common. They can be very bad for cars. We are trying to find all the potholes so we can fix them.

Crack is a line on the road. Sometimes we see one crack. Sometimes we see a lot of cracks. This is what happens when the road starts to get old. If we do not fix the cracks they get bigger. The road gets worse.

Alligator Cracking is a kind of crack that looks like the skin of an alligator. It happens when the road gets tired from all the cars driving on it. This means the road is getting weak.

Ravelling is when the road starts to fall. This happens when the little rocks on the road start to come. The road gets rough. It is hard to drive on.

Rutting is when the road gets grooves in it from all the cars driving on it. This makes it hard to drive. It makes water stay on the road. We do not like rutting because it is bad for the road and it is bad, for cars.

Roadside Object Classes

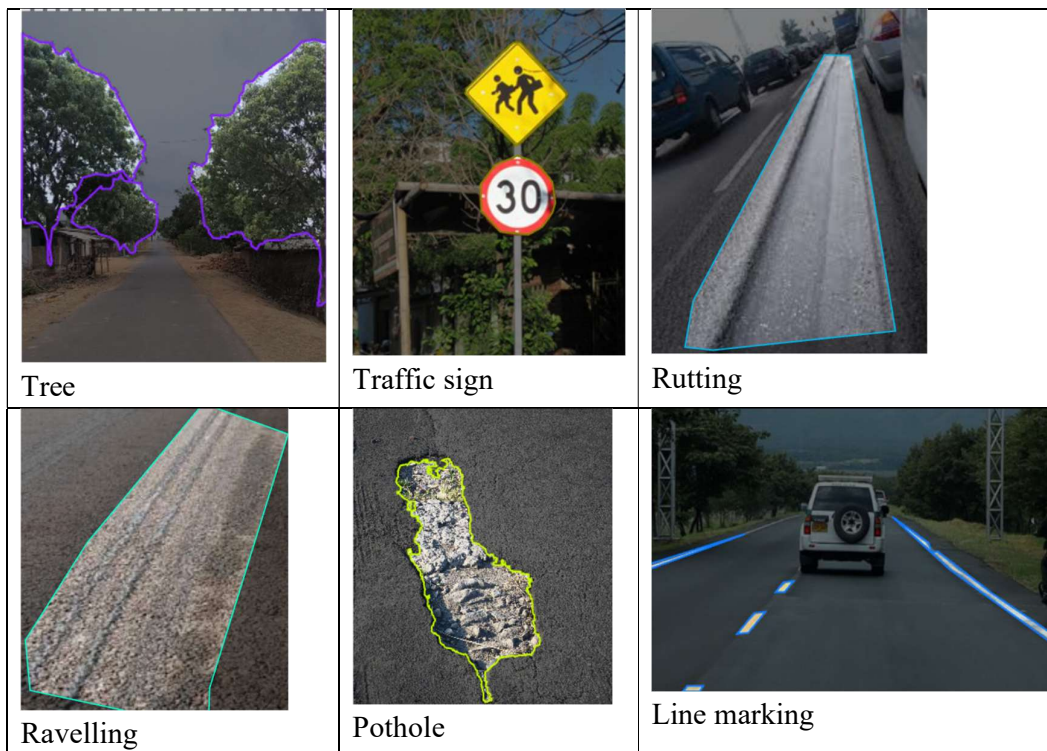
These classes capture infrastructure elements visible in road survey footage that are relevant to road condition assessment and survey documentation.

Guard Rails are longitudinal safety barriers installed along the edges of roads to prevent vehicles from leaving the carriageway.

Line Marking refers to painted markings on the road surface including lane dividers, centre lines, and edge lines that guide traffic flow.

Traffic Sign refers to regulatory, warning, or informational signs installed alongside or above the road.

Tree refers to trees present along the roadside, which are relevant for road corridor assessment and obstruction analysis.



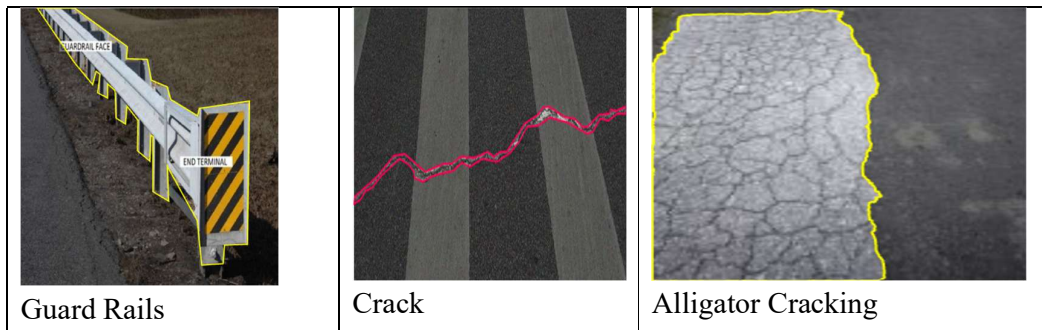


Figure 2.2- Classes used in dataset

2.5 Annotation Process

The annotation process is when we label each image by drawing boundaries around every instance of each class. For this project we used instance segmentation annotation, which means we drew a polygon mask around each object of just a simple box around it. This way we get precision down to the pixel level, which's necessary for training a segmentation model like YOLOv11l-seg and YOLO26l-seg.

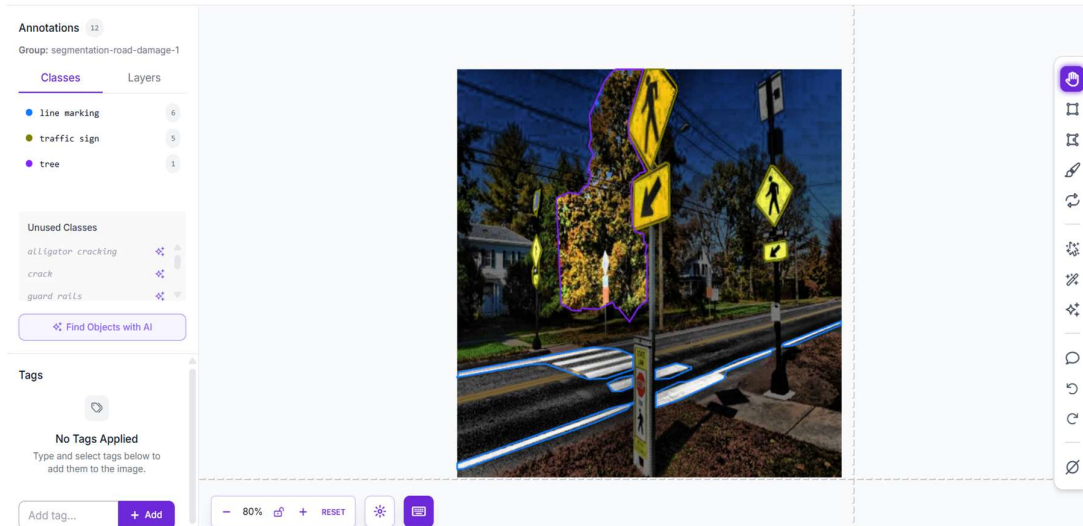
We did the annotation over the course of the internship going through the collected images one by one.

2.5.1 Annotation Tool — Roboflow

We chose Roboflow as our tool for annotating and managing the dataset. It has an interface in the browser that lets us draw polygons to annotate the images and it can export the annotations in the YOLO segmentation format, which works perfectly with the Ultralytics training pipeline.

Here is how we annotated the images in Roboflow. First we uploaded all the collected images to a Roboflow project. Then we defined the class labels in the project, which were the nine classes we listed in Section 2.4. For each image the people annotating used the polygon tool to draw the boundary of every instance of each class that they could see. When an image was fully annotated we marked it as complete. Sent it to the review queue.

Roboflow also has a feature to check the health of the dataset, which shows us how the classes are distributed how images we have annotated and some statistics, about the annotations. We used this feature to keep track of our progress and find classes that did not have examples.



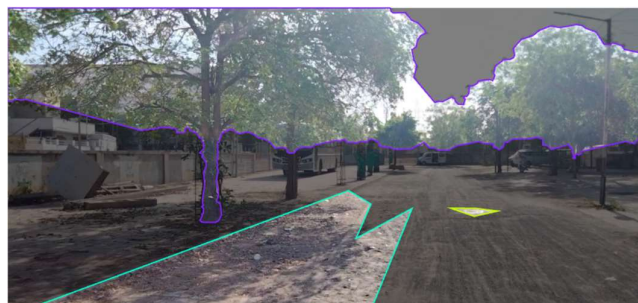
Screenshot 2.1-Roboflow annotation interface

The final dataset was exported from Roboflow in a format that works with YOLOv11 segmentation. This format gives us one text file per image. Each text file has the class index and adjusted polygon coordinates for every annotated object. We also got a data.yaml file that defines the class names and where to find the dataset.

2.5.2 Role of SAM in Annotation

The Segment Anything Model or SAM is a model made by Meta AI. It can create object masks with simple instructions like a click or a box around an object. We used SAM to make it faster to create polygon masks. This was especially helpful for damage like alligator cracking and ravelling. Drawing polygons, around these by hand takes a lot of time.

In our process the person annotating would draw a box or click on the damaged area. Then SAM would create a mask that fits the actual edge of the damage. They could then accept this mask. Adjust it by hand before saving it as the final annotation. This made it much quicker to annotate damage that is not easily shaped while keeping the masks precise



Screenshot 2.2-SAM-assisted annotation in Roboflow showing auto-generated mask on tree

2.6 Dataset Splitting

Following annotation, the complete dataset was split into three subsets — training, validation, and test — following standard practice in machine learning. The split was performed within Roboflow prior to export.

Split	Purpose	Images
Train	Model learning	3614
Validation	Hyperparameter tuning and early stopping	80
Test	Final unbiased evaluation	39
Total		3733

Table 2.2 — Dataset Split Distribution

The training set was used exclusively for weight updates during model training. The validation set was used at the end of each epoch to compute mAP and loss metrics, which guided early stopping and learning rate scheduling. The test set was held out entirely during training and used only for final evaluation to provide an unbiased estimate of model performance on unseen data.

Care was taken to ensure that all three splits contained representative samples of each class, particularly for rare classes such as rutting, where the limited number of samples made balanced splitting more critical.

2.7 Dataset Challenges and Limitations

Several challenges were encountered during dataset preparation that directly impacted annotation quality and dataset composition.

Class Imbalance

As shown in Table 2.1, the dataset is highly imbalanced. Tree instances (1,680) outnumber rutting instances (58) by a ratio of approximately 29:1. This reflects the natural occurrence frequency of these classes on Indian roads but creates a significant training challenge, as the model tends to learn dominant classes more effectively than rare ones.

Visual Similarity Between Classes

Several damage classes share similar visual characteristics, making annotation boundaries ambiguous. Alligator cracking and crack regions often appear adjacent to each other. Ravelling and alligator cracking can appear visually similar in low-resolution or poorly lit images. These ambiguities occasionally led to inconsistency in annotation boundaries between different images annotated at different times.

Varying Image Quality

Frames extracted from video footage exhibited varying levels of motion blur, particularly at higher vehicle speeds. Additionally, lighting conditions varied across collection sessions, with some frames captured under direct sunlight causing glare on the road surface, and others captured in shadow or overcast conditions. These variations affected annotation clarity and model training consistency.

Limited Rare Class Samples

Despite supplementing the dataset with internet-sourced images, rutting remained severely underrepresented with only 58 annotated instances. This was primarily because rutting is less visually distinct and less commonly documented in publicly available road damage imagery.

No GPS or Location Metadata

The dataset did not include GPS coordinates or location metadata associated with individual images. This means spatial mapping of detected damage — which would significantly enhance the utility of the system — was not possible within the scope of this project.

3. System Architecture

The Road Condition Detection project is made up of parts that work together to turn a raw video of a road into a clear report of the damage. These parts are organized in a way like a pipeline, where each part does its job and then passes its work to the next part. This way if we need to make a change to one part it does not affect the parts.

The whole process starts with taking in a video then preparing it finding the damage tracking it measuring it and adding notes. Finally it shows the results to the user in a dashboard made with Streamlit.

3.1 Overall Pipeline Overview

The system takes the video. Breaks it down into individual frames, which are then processed one by one. The process can be divided into five steps:

- 3 The user gives us a video of the road either by uploading it or giving us the file directly. We use OpenCV to open the video and read the frames one by one.
- 4 We use a model called YOLOv11-seg to look at each frame and find the damage. The model tells us what it found, how sure it is and where exactly the damage is.
- 5 We filter out the damage that is not important and make sure the damage we found is actually on the road. If we find the type of damage in the same place we combine them.
- 6 We use another model called ByteTrack to track the damage over time so we can count how times it appears. We also measure the size of the damage using a formula.
- 7 Finally we add notes to each frame to show where the damage is, what type it is and how sure we are. We then save the frames as a video and send the results to the dashboard.

The input part of the system is responsible for taking in the video and preparing it for the rest of the process. We use OpenCV to handle the video and get information like the width, height and frames, per second. To make the process faster we do not look at every frame. Rather every other frame or every few frames. The user can decide how often we look at the frames.

3.2 Input Module

The input module gets the road survey video. Makes each frame available to the rest of the pipeline. Video input is handled using OpenCV's video capture tool. This tool supports video formats like MP4, AVI, MOV and MKV. When the video is opened the module reads details like frame width, height and frames per second. These details are used later for writing the output video and calculating the GSD. To balance processing speed and

detection coverage the system skips some frames. It does not process every frame because consecutive frames, in a road survey video are almost the same. Instead it processes every Nth frame. The skip interval N can be set by the user. By default N is 2 which means every other frame is processed. This approach cuts the processing time in half with loss of detection coverage. The input module and frame skipping strategy help the system work efficiently. The road survey video is processed in a way that balances speed and accuracy.

Parameter	Default Value	Description
skip_frames	2	Process every Nth frame
imgsz	800	Inference image size (pixels)
Output FPS	source FPS / skip_frames	Output video frame rate

Table 3.1- Input Module Parameters

3.3 Preprocessing

Before a frame is sent to the YOLOv11l-seg model we do some preparation. The Ultralytics inference pipeline takes care of a things for us. It resizes the input frame to a size that makes sure the picture does not get distorted. It adjusts the pixel values to be between 0 and 1. We do not do any preparation like making the picture brighter or sharper because the YOLOv11l-seg model was trained on pictures and can handle different lighting.

One thing we do before running the YOLOv11l-seg model is create a road mask. This is explained in Section 3.6. We do this at the time as we run the YOLOv11l-seg model and we use the result to filter out things we do not want after the YOLOv11l-seg model has finished.

3.4 Detection and Segmentation Module

This module is a part of our system. It takes frames. Returns the results of what it found including what type of thing it is how sure it is, where it is and a mask that shows exactly where the thing is. We use the YOLOv11l-seg model for this, which we access using the Ultralytics Python API. We load the YOLOv11l-seg model once. Then we use it for all frames so we do not have to load it every time.

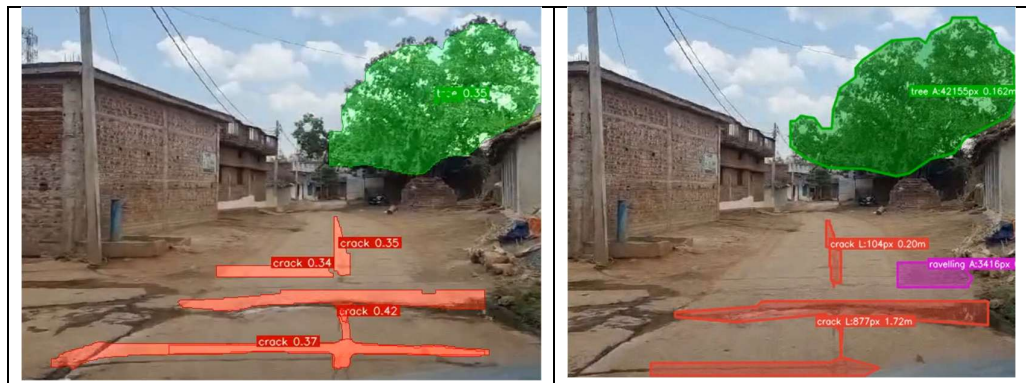
For each frame the YOLOv11l-seg model does one. Returns a list of things it found. Each thing it found includes:

- What type of thing it is, which is one of nine types
- A mask that shows where the thing is
- How big the thing is in pixels and square meters

Mask Alignment Fix

We have to be careful when we resize the masks that the YOLOv11l-seg model produces. The YOLOv11l-seg model makes masks that're smaller than the original frame so we have to make them bigger again. If we just use the way to make them bigger the edges of the mask do not line up with the edges of the thing in the picture. To fix this we use a way to make the mask bigger. Then we make it a simple mask by setting a threshold of 0.5. This way the masks line up perfectly with the things, in the picture and the YOLOv11l-seg model results are more accurate.

```
mask = cv2.resize(raw_mask, (frame_width, frame_height),  
                  interpolation=cv2.INTER_LINEAR)  
binary_mask = (mask > 0.5)
```



Screenshot 3.1-Comparison: distorted mask vs corrected mask

Confidence and IoU Thresholds

Detections below a confidence threshold of 0.30 are discarded before any further processing. A non-maximum suppression IoU threshold of 0.5 is applied internally by the Ultralytics model to remove duplicate detections of the same object.

3.5 Measurement Module

The measurement module computes physical dimensions of detected damage from pixel-level segmentation masks. Two types of measurement are performed: area estimation for pothole and ravelling, and length estimation for crack.

3.5.1 Area Estimation — GSD Method

Area estimation uses the Ground Sampling Distance (GSD) photogrammetric formula to convert a pixel area measurement into a real-world area in square metres.

GSD represents the real-world distance corresponding to one pixel in the image and is calculated as:

$$\text{GSD (m/pixel)} = (H \times Sh) / (f \times ih)$$

Where:

- H = camera height above road surface in metres
- Sh = sensor height in millimetres
- f = focal length in millimetres
- ih = image height in pixels

The real-world area of a detected damage region is then:

$$\text{Area (m}^2\text{)} = \text{pixel_count} \times \text{GSD}^2$$

Where pixel_count is the number of True pixels in the binary segmentation mask of the detected instance.

Both the pixel area and the computed metre-squared area are reported in the output for each detected instance of pothole and ravelling.

Parameter	Default Value
Focal length (f)	4.3 mm
Sensor height (Sh)	3.6 mm
Camera altitude (H)	1.5 m

Table 3.2 — Default Camera Parameters Used for GSD Calculation

3.6 Output Module

The output module is responsible for rendering the annotated video and compiling the final summary data structure.

Visual Annotation

For each processed frame, the following visual elements are rendered on the output frame:

- Semi-transparent filled mask overlaid on each detected object, coloured according to its class using the defined colour scheme

- Contour outline drawn around each mask boundary for visual clarity
- Class label and confidence score displayed at the mask centroid, with a solid colour background rectangle for readability
- No bounding boxes are drawn, keeping the output visually focused on segmentation masks

Output Video

Annotated frames are written sequentially to an output video file using OpenCV's VideoWriter with the mp4v codec. Since mp4v-encoded video is not directly playable in most browsers and media players, the output file is re-encoded using FFmpeg with the H.264 codec before being made available for download. The re-encoding is performed with a constant rate factor of 23 and the fast preset, balancing output quality with encoding speed.

Summary Data Structure

In addition to the annotated video, the output module compiles a structured summary dictionary containing:

- Total frames processed
- Unique object counts per tracked class
- Area summary per class including total, average, maximum, and minimum values in both pixels and square metres
- Length summary per crack class in both pixels and metres
- Camera parameters and formulas used for measurement
- Per-frame statistics for chart generation in the dashboard

4. Training

Training is the process by which each model learns to perform its assigned segmentation task from the annotated dataset. This chapter describes the complete training process for both models used in this project — the YOLOv11l-seg road surface mask model and the YOLOv26l-seg road damage detection model. It covers the training environments used, the transfer learning approach, the progression through model sizes for the damage detection task, hyperparameter configuration, augmentation strategy, the two-stage training approach, training curve observations, and practical challenges encountered.

4.1 Training Environment

Training deep learning segmentation models requires significant GPU memory and compute. This project was conducted as a student internship without access to dedicated on-premise GPU hardware, so cloud-based GPU platforms were used throughout. Different platforms were used at different stages of the project depending on the model being trained and the resources required.

Platform Evaluation

Three platforms were evaluated and used across different stages of the project.

Google Colab provides free access to a single NVIDIA T4 GPU with 15 GB of VRAM. Sessions are subject to unpredictable time limits of approximately two to three hours, after which the runtime is terminated. While suitable for short inference tests and debugging, Colab was not reliable for full training runs due to the session cutoff risk.

Kaggle provides free access to two NVIDIA Tesla T4 GPUs with 15.6 GB of VRAM each, totaling 30 GB per session. Sessions run for up to twelve hours without interruption and notebooks along with output weights are automatically saved. Kaggle was used for preliminary training experiments with YOLOv11s-seg and YOLOv11m-seg on the road damage dataset, and for training the YOLOv11l-seg road surface mask model.

Ultralytics Hub is a managed cloud training platform provided by Ultralytics that connects directly to the Ultralytics model and dataset ecosystem. It provides access to high-performance GPU instances on demand at an hourly cost. The final road damage detection model — YOLOv26l-seg — was trained on Ultralytics Hub using an NVIDIA RTX PRO 6000 GPU, which completed training in one hour at a total compute cost of \$2.11.

Platform	GPU	VRAM	Session Limit	Cost	Used For
Google Colab	T4 ×1	15 GB	~2–3 hrs	Free	Inference testing

Kaggle	T4 ×2	15.6 GB ×2	12 hrs	Free	YOLOv11s, v11m, road mask
Ultralytics Hub	RTX PRO 6000	48 GB+	No limit	\$2.09/hr	YOLOv26l final model

Table 4.1 — GPU Platform Comparison

Model	Purpose	Platform	GPU	Duration
YOLOv11s-seg	Damage detection experiment	Kaggle	T4	~2 hrs
YOLOv11m-seg	Damage detection experiment	Kaggle	T4	~4 hrs
YOLOv11l-seg	Road surface mask	Kaggle / Colab	T4	Multiple sessions
YOLOv26l-seg	Road damage detection (final)	Ultralytics Hub	RTX PRO 6000	1 hour

Table 4.2 — Per-Model Training Environment Summary

Run Information			Completed
Training timing and environment details			
Start Time Jun 16, 2026, 09:35 AM UTC	Runtime 1h	Compute Cost \$2.11 RTX PRO 6000 - \$2.09/hr	
Ultralytics 8.4.67	Hostname lambda-scalar	Environment Docker	
OS Linux-5.15.0-173-generic-x86_64-with-glibc2.39	Python Version 3.12.3	CPU AMD EPYC 9655 96-Core Processor	
GPU RTX PRO 6000	Parent Model yolo26l-seg	Git https://github.com/ultralytics/ultralytics main	
Command. Requires ultralytics>=8.4.61			
yolo train device=7 model=ul://ultralytics/yolo26/yolo26l-seg data=ul://anshika-tiwari/datasets/final project=anshika-tiwari/example-project name=exp-4 epochs=100 batch-size=16			

Screenshot 4.1- Ultralytics Hub Run Information page

Ultralytics Hub Environment Details

The final YOLOv26l-seg training run on Ultralytics Hub operated in the following environment:

Component	Specification
Platform	Kaggle Notebooks
Model	YOLO26l-seg
GPU	NVIDIA Tesla T4
Operating System	Windows
Python Version	3.10
Ultralytics Version	8.4.67
Training Epochs	100
Optimizer	AdamW
Batch Size	Auto
Component	Specification
Platform	Ultralytics Hub

Table 4.3 — Ultralytics Hub Training Environment

Kaggle Environment Details

Preliminary experiments and road mask model training used the following Kaggle environment:

Component	Specification
Platform	Kaggle Notebooks
GPU	NVIDIA Tesla T4
VRAM	15.6 GB
Python	3.12
Ultralytics	8.4.67
Training mode	Single GPU

Table 4.4 — Kaggle Training Environment

4.2 Pretrained Weights and Transfer Learning

All training runs in this project used transfer learning from COCO-pretrained weights rather than training from random initialization. Transfer learning is a technique in which a model trained on a large general-purpose dataset is used as the starting point for training on a smaller domain-specific dataset.

Both YOLOv11l-seg and YOLOv26l-seg are pretrained on the COCO dataset, which contains approximately 118,000 training images across 80 object categories. Through this pretraining, the backbone network learns general visual features — edges, textures, gradients, shapes, and object parts — that transfer effectively across a wide range of visual recognition tasks including road damage detection.

When fine-tuning on the road damage dataset, the pretrained backbone weights provide a strong feature extraction foundation. The model does not need to relearn low-level visual features from scratch, allowing training to focus on learning the specific visual characteristics of road damage classes. This is particularly important given the limited dataset size of approximately 1,300 images, where training from random initialisation would almost certainly result in severe overfitting or underfitting.

For both models, the detection and segmentation head weights were reinitialised to match the number of output classes in the respective training task — nine classes for the damage detection model and one class for the road mask model — replacing the original 80-class COCO head.

For the YOLOv26l-seg model trained on Ultralytics Hub, pretrained weights were loaded directly from the Ultralytics model repository using the model identifier `ul://ultralytics/yolo26/yolo26l-seg`, which automatically fetched the COCO-pretrained checkpoint before beginning fine-tuning on the road damage dataset.

4.3 Model Progression

The road damage detection model was not developed in a single step. A systematic progression through increasing model sizes was followed to understand the relationship between model capacity and detection performance on this specific dataset before committing to the final model. The road surface mask model followed a separate, simpler training process.

4.3.1 Road Damage Model: YOLOv11s → YOLOv11m → YOLOv11l → YOLOv26l

Four successive training experiments were conducted for the road damage detection task, each using a more capable model than the previous.

YOLOv11s-seg — Initial Experiment

The first training run used YOLOv11s-seg, the small variant of the YOLOv11 segmentation family with approximately 9.4 million parameters. This was chosen as the starting point because it trains fastest, allowing rapid verification that the dataset configuration, data.yaml paths, and training loop were functioning correctly before committing to longer runs.

YOLOv11s was trained on Kaggle using a single T4 GPU. Results showed limited detection performance across most damage classes. The small model capacity was insufficient to simultaneously learn discriminative features for nine classes from a 1,300-image dataset. Rare classes such as rutting showed near-zero detection performance.

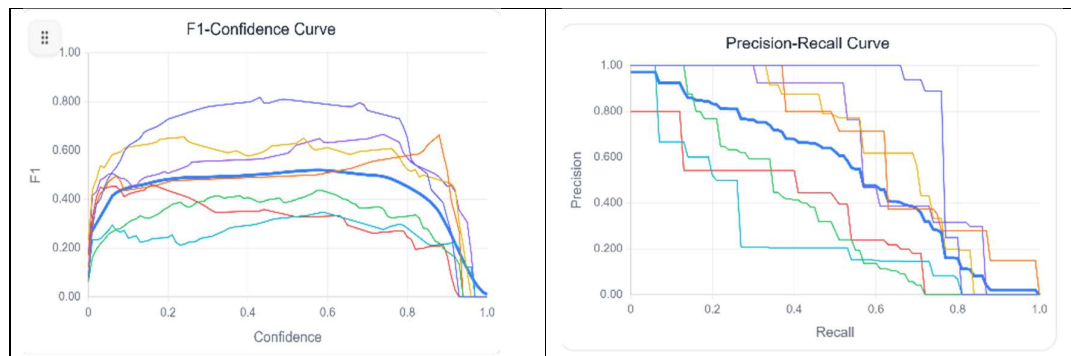
YOLOv11m-seg — Intermediate Experiment

The second run used YOLOv11m-seg with approximately 20.1 million parameters — more than double the capacity of the small model. Training was again conducted on Kaggle T4.

YOLOv11m showed meaningful improvement over YOLOv11s across most classes. High-frequency classes such as tree, guard rails, and line marking showed noticeably improved confidence and recall. Damage classes including pothole and crack also improved. Rare classes such as rutting and alligator cracking continued to underperform due to limited training samples.

YOLOv11l-seg — Third Experiment

The third run used YOLOv11l-seg with approximately 25.3 million parameters, trained on Kaggle T4. This run applied the full augmentation strategy and two-stage training approach described in Sections 5.5 and 5.6. YOLOv11l showed further improvement over YOLOv11m, confirming that increasing model capacity was yielding consistent gains on this dataset.



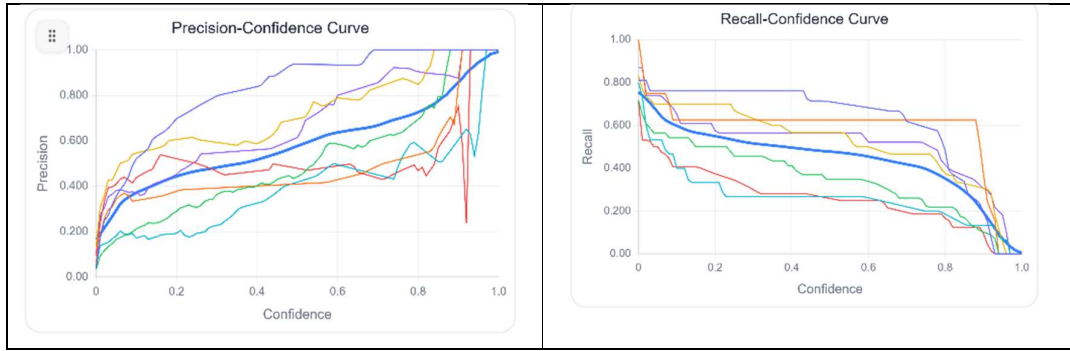


Figure 4.1-Training curves (YOLOv11-seg)

4.3.2 YOLOv26l-seg — Final Damage Detection Model

The final and best-performing damage detection model used YOLOv26l-seg, trained on Ultralytics Hub. Based on the consistent gains observed across YOLOv11s, YOLOv11m, and YOLOv11l, the decision was made to train the largest and most capable available model on the most powerful available GPU infrastructure.

The training completed in one hour on an NVIDIA RTX PRO 6000 GPU. The RTX PRO 6000 provides substantially more compute than the Kaggle T4, enabling the large YOLOv26l model to train to convergence in a fraction of the time that would have been required on free GPU resources.

The final model achieved mAP50 of 78.8%, mAP50-95 of 50.4%, Precision of 79.5%, and Recall of 73.0% on the validation set.



Screenshot 4.2- Performance Metrics of the Trained YOLO26l-seg Model

4.3.3 Road Surface Mask Model — YOLOv11l-seg

The road surface mask model was trained separately from the damage detection pipeline. Its purpose is to generate a binary segmentation of the drivable road surface from each video frame, which is used downstream to filter damage detections to the road area only.

This model uses YOLOv11l-seg trained on a separately annotated dataset where the road surface region was labelled as a single class. The training was conducted on ultralytics.

Parameter	Value
Total Training Images	2,252
Total Annotated Road Masks	2,847
Model Architecture	YOLO11 Instance Segmentation
Training Epochs	100
Image Size	640 × 640 pixels
Final mAP@50	84.6%

Table 4.5- Parameters of Road Mask Model



Figure 4.2- Image with road mask

4.4 Hyperparameter Configuration

Hyperparameters govern the training process and significantly influence final model quality. The hyperparameters for each training run were selected based on Ultralytics recommended defaults, adjusted for the characteristics of the road damage dataset.

Hyperparameter	YOLOv11s	YOLOv11m
Epochs	100	150
Image size	640	640
Batch size	16	16
Optimizer	AdamW	AdamW
lr0	0.001	0.001
Device	Kaggle T4	Kaggle T4
Augmentation	Standard	Standard

Table 4.6- Hyperparameter Configuration: YOLOv11s and YOLOv11m Experiments

Hyperparameter	YOLOv11l	YOLOv26l
Epochs	300	100
Image size	640	640
Batch size	8	16
Optimizer	AdamW	AdamW
lr0	0.001	0.01
weight_decay	0.0005	0.0005
warmup_epochs	5	3
cos_lr	True	False
label_smoothing	0.1	0.1
Early stopping patience	50	na
Device	Kaggle T4	RTX PRO 6000

Table 4.7- Hyperparameter Configuration: YOLOv11l (Kaggle) and YOLOv26l (Ultralytics Hub)

Optimizer — AdamW

AdamW was used for all Kaggle-based training runs. For small datasets, AdamW converges faster and more reliably than SGD because it adapts the learning rate individually for each parameter based on gradient history, requiring less manual learning rate tuning.

Cosine Learning Rate Schedule

A cosine annealing schedule decays the learning rate from the initial value to the final value following a cosine curve, allowing large weight updates early in training and fine adjustments near convergence. This schedule was applied for all YOLOv11-family runs on Kaggle.

Label Smoothing

Label smoothing with a factor of 0.1 was applied during YOLOv11l training on Kaggle, replacing hard binary targets with softened values of 0.1 and 0.9. This reduces overconfidence in predictions and improves generalisation on imbalanced datasets where the model might otherwise memorise dominant class patterns.

4.5 Data Augmentation Strategy

Data augmentation is critical for a dataset of only 1,300 images. Without augmentation the model would quickly overfit to the training set and perform poorly on unseen road survey footage. The augmentation strategy was designed to address three specific

challenges: small dataset size, severe class imbalance, and the visual variability of real-world Indian road survey conditions including different lighting, surface colours, road types, and camera positions.

Copy-Paste Augmentation

Copy-paste augmentation randomly selects object instances from other training images and pastes their segmentation masks and image regions into the current training image. This is particularly effective for rare classes — rutting and alligator cracking instances can be copied from the few images where they appear and pasted into many other images, multiplying effective training samples for underrepresented classes without additional annotation effort. Copy-paste was applied with probability 0.4 during Stage 1 and 0.5 during Stage 2.

Geometric Augmentations

Augmentation	Value	Purpose
Rotation	$\pm 10^\circ$	Camera angle variation
Translation	$\pm 15\%$	Camera position shift
Scale	$\pm 60\%$	Distance variation
Shear	$\pm 2^\circ$	Perspective distortion
Perspective warp	0.0003	Road geometry variation
Vertical flip	0.3 probability	Survey direction variety
Horizontal flip	0.5 probability	Left/right road symmetry
Multi-scale resize	Enabled	Scale invariance

Table 4.8- Geometric Augmentation Parameters

Photometric Augmentations

Augmentation	Value	Purpose
Hue shift (hsv_h)	0.02	Colour cast simulation
Saturation (hsv_s)	0.7	Wet vs dry road surface
Brightness (hsv_v)	0.5	Day, shadow, overcast lighting
Random erasing	0.4	Occlusion simulation
MixUp	0.1	Soft image blending

Table 4.9 — Photometric Augmentation Parameters

5.Implementation

This chapter describes the complete software implementation of the Road Condition Detection system. It covers the development environment, the inference pipeline architecture using two separate YOLO models, and the detailed implementation of each processing component including mask alignment and rendering, object tracking, overlap merging, road surface masking, area and length measurement, and video output encoding. All implementation was carried out in Python using the libraries and tools listed in the technology stack.

5.1 Development Environment

The inference pipeline and dashboard were developed and tested primarily on Google Colab, which provided a convenient browser-based Python notebook environment with free GPU access suitable for running and iterating on the inference code against test videos. Once the pipeline was stable, the inference logic was consolidated into a standalone Python module — `inference.py` — and the dashboard was built as a separate file — `dashboard.py` — that imports the inference module directly.

Component	Specification
Development platform	Google Colab
Testing GPU	NVIDIA T4
Language	Python 3.12
Deep learning framework	PyTorch
Model API	Ultralytics 8.4.67
Computer vision	OpenCV
Mask processing	scikit-image
Data handling	NumPy, Pandas
Dashboard framework	Streamlit
Video encoding	FFmpeg

Table 5.1 — Development Environment Specifications

The project codebase consists of two primary files that together constitute the complete deployable system:

- `inference.py` — the complete backend module containing the two-model detector pipeline, ByteTracker, road mask generator, overlap merger, measurement functions, and video processing logic
- `dashboard.py` — the Streamlit dashboard that imports and uses the inference module to provide an interactive browser-based interface

All supporting utility functions are self-contained within `inference.py`, making the backend portable and importable without additional dependencies beyond the listed libraries.

5.2 Inference Pipeline

The inference pipeline is implemented as the `RoadDamageDetector` class within `inference.py`. This class encapsulates both YOLO models, the `ByteTracker` for each tracked class, and the per-session accumulation of measurements. Instantiating the class loads both models into memory once, after which the detector can process any number of frames or videos without reloading either model.

Two-Model Sequential Architecture

A key architectural feature of this implementation is the use of two separate YOLO models operating sequentially on each frame. This is a deliberate design decision that separates the road surface understanding task from the damage detection task, allowing each model to specialise in its own domain.

The two models operate as follows on each frame:

Model 1 — YOLOv11l-seg (Road Surface Mask)

The road surface mask model runs first on each frame. It was trained to segment the drivable road surface as a single class, producing a binary mask that identifies which pixels in the frame constitute road surface. This mask is used downstream to spatially constrain damage detections to the road area only, preventing false positives on non-road surfaces such as footpaths, embankments, and vegetation.

Model 2 — YOLOv26l-seg (Road Damage Detection)

The damage detection model runs second on the same frame. It was trained to detect and segment nine classes of road damage and roadside objects simultaneously. Its output — a set of per-instance segmentation masks with class labels and confidence scores — is filtered using the road surface mask produced by Model 1 for road-damage-specific classes.

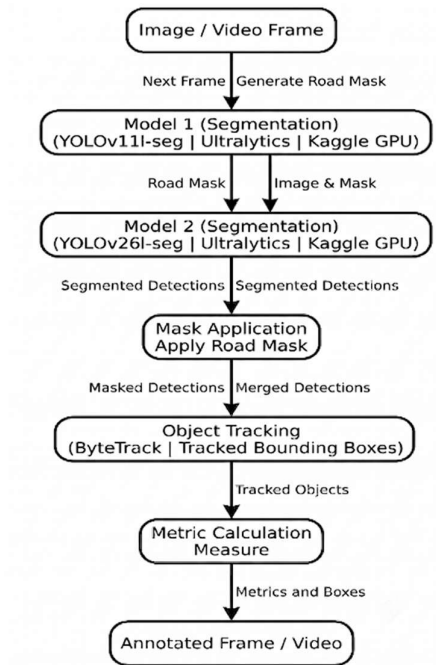


Figure 5.1- Block Diagram of the Proposed Video Processing Framework

Public Interface

The RoadDamageDetector class exposes three primary public methods:

- **process_frame(frame)** — Processes a single video frame (NumPy BGR array) and returns the annotated frame together with the detection statistics for that frame.
- **process_video(source, output_path, skip_frames)** — Processes an entire input video, performs road segmentation, damage detection, tracking, and analytics, and generates an annotated output video along with a summary of detected road damages.
- **process_image(image_path, save_path)** — Processes a single input image and saves the annotated output image.

The **process_frame()** method forms the core of the complete processing pipeline. Both **process_video()** and **process_image()** internally invoke this function for each frame or image.

The execution sequence of **process_frame()** is as follows:

1. Read the input frame captured using OpenCV.
2. Pass the frame to the **YOLO11 Road Segmentation Model (exp-2.pt)** to generate a binary road mask.

3. Pass the original frame to the **YOLO Road Damage Detection Model (best.pt)** to detect road defects using instance segmentation.
4. Resize the predicted segmentation masks to the original frame dimensions whenever required.
5. Apply the binary road mask using a logical AND operation to retain only detections lying within the segmented road surface.
6. Filter invalid detections by removing very small potholes and low-quality segmented regions below the predefined threshold.
7. Draw coloured segmentation masks and class labels for all valid detections on the output frame.
8. Compute area measurements for area-based defects such as potholes, ravelling, rutting, and alligator cracking using the Ground Sampling Distance (GSD).
9. Perform multi-object tracking using **ByteTrack** to maintain persistent object IDs and compute unique counts of potholes and trees across the video.
10. Compile frame statistics, including object counts and measured defect areas, and return the annotated frame for video generation

5.3 Mask Alignment and Rendering

Correct mask alignment was one of the most critical implementation details identified during development. Misaligned masks — where the coloured overlay does not coincide with the actual detected object in the frame — were one of the primary visual quality issues observed during early testing of the inference pipeline.

Root Cause of Misalignment

The YOLOv26l-seg model produces segmentation masks at the resolution of its internal feature map, which corresponds to the inference image size rather than the original frame resolution. When the model processes a frame of resolution 1136×640 at inference size 800, the raw mask tensor dimensions are smaller than the original frame. Additionally, the Ultralytics inference pipeline applies letterboxing — padding the frame to a square before resizing to the inference size — which introduces an additional spatial offset if not accounted for during mask resize.

If the raw masks are resized to the frame dimensions using nearest-neighbour interpolation, the resulting masks appear blocky with straight-line artefacts along boundaries, and the mask boundaries do not align with the actual object edges in the original frame. This was the exact issue observed in early testing — masks appeared shifted upward or to the right relative to the detected objects.

Fix Implemented

The misalignment was resolved by using bilinear interpolation for mask resizing followed by thresholding at 0.5 to produce a clean binary mask. Bilinear interpolation computes each output pixel as a weighted average of the four nearest input pixels, producing smooth, well-aligned boundaries.

```
python

resized      = cv2.resize(raw_mask, (frame_width, frame_height),
                           interpolation=cv2.INTER_LINEAR)

binary_mask = (resized > 0.5)
```

This single change eliminated the distortion and straight-line artefacts observed in initial testing and produced masks that accurately conform to object boundaries in the original frame.

Mask Rendering

Each binary mask is rendered on the output frame as a semi-transparent coloured overlay. A blank colour layer of the same dimensions as the frame is created, filled at the mask region with the assigned class colour, and blended with the original frame using OpenCV's `addWeighted` function at an alpha value of 0.45.

```
python

colored = np.zeros_like(frame, dtype=np.uint8)

colored[mask] = class_color

out = cv2.addWeighted(out, 1.0, colored, 0.45, 0)
```

Following the fill, the mask contour is extracted using `cv2.findContours` and drawn as a solid outline with a line thickness of 2 pixels. This contour provides a clear visual boundary around each detected region, making the segmentation extent readable even where the transparent fill blends into a similarly coloured road surface.

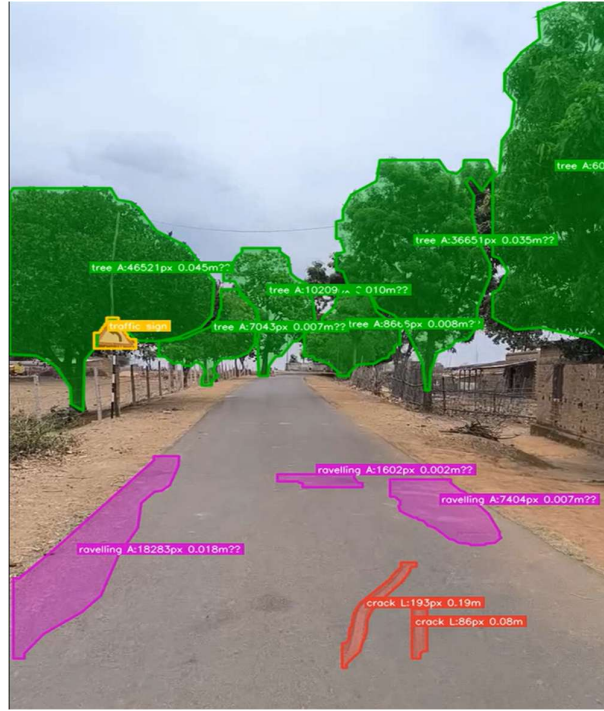
The class label and confidence score are rendered as text positioned at the mask centroid, the mean x and y coordinates of all True pixels in the binary mask with a solid rectangular background in the class colour for readability against any background. No bounding boxes are drawn anywhere in the output, keeping the visual presentation focused entirely on the segmentation masks.

Parameter	Value
Fill transparency (alpha)	0.45
Contour thickness	2 px
Label position	Mask centroid
Label font	FONT_HERSHEY_SIMPLEX
Label scale	0.45
Label text colour	White (255, 255, 255)
Label background	Class colour solid rectangle
Bounding box	Not drawn
ID display	Disabled

Table 5.2 — Mask Rendering Parameters

Class	BGR Value	Colour
Alligator cracking	(0, 0, 255)	Red
Crack	(0, 0, 0)	Black
Guard rails	(255, 255, 255)	White
Line marking	(255, 255, 0)	Cyan
Pothole	(255, 0, 0)	Blue
Ravelling	(255, 0, 255)	Magenta
Rutting	(0, 255, 255)	Yellow
Traffic sign	(0, 255, 0)	Green
Tree	(34, 139, 34)	Forest green

Table 5.3 — Class Colour Assignments (BGR)



Screenshot 5.1- Output video with mask rendering

5.4 Multi-Object Tracking Using ByteTrack

Object tracking in the proposed system is performed using the **ByteTrack** multi-object tracking algorithm integrated within the Ultralytics YOLO framework. ByteTrack assigns a unique tracking ID to each detected object and maintains that ID across consecutive video frames. This enables the system to distinguish between newly appearing objects and previously detected objects, preventing duplicate counting of the same physical road defect.

Without object tracking, the same pothole or roadside object would be detected independently in every processed frame, resulting in multiple counts of a single physical object. By maintaining persistent object identities, ByteTrack ensures that each pothole and tree is counted only once throughout the video, improving the reliability of the generated road condition statistics.

Unlike traditional centroid-based tracking methods, ByteTrack performs data association by combining object confidence scores with spatial information, allowing it to maintain object identities even when temporary detection failures or partial occlusions occur. This makes the tracker more robust for road survey videos where vehicle motion and changing viewpoints are common.

Tracking Procedure

For each processed frame, the following operations are performed:

1. The road damage detection model generates segmented detections for all target classes.
2. The detected objects are passed to the ByteTrack tracking algorithm.
3. ByteTrack associates current detections with existing object tracks and assigns a persistent tracking ID to each object.
4. New objects entering the scene are assigned new unique IDs, while previously tracked objects retain their existing IDs.
5. The tracking IDs are internally maintained throughout the video to compute unique object counts for potholes and trees.
6. The final tracking information is combined with the segmentation masks and area measurements to generate the annotated output frame and summary statistics.

The tracking IDs are primarily used for internal analytics and unique object counting. Although ByteTrack maintains persistent IDs for every tracked object, these IDs may optionally be displayed or hidden in the output video depending on the visualization settings of the application

Parameter	Value	Purpose
Tracking Algorithm	ByteTrack	Multi-object tracking and ID assignment
Configuration File	bytetrack.yaml	Default tracking configuration provided by Ultralytics
Tracking Mode	Persistent (persist=True)	Maintains object IDs across consecutive frames
Tracked Classes	Pothole, Tree	Classes used for unique object counting
ID Assignment	Automatic	Assigns and maintains unique IDs for detected objects
ID Display	Configurable	IDs can be displayed or hidden in the output video

Table 5.4 – ByteTrack Configuration

Anti-Flickering Design

The proposed system benefits from the built-in track management mechanism of the ByteTrack algorithm. During video processing, an object may occasionally fail to be detected for one or more consecutive frames due to motion blur, partial occlusion, lighting variations, or temporary segmentation errors. Instead of immediately assigning a new tracking ID when the object reappears, ByteTrack attempts to associate the new detection with the existing object track using its internal data association strategy.

This mechanism reduces tracking flicker by maintaining consistent object identities across consecutive frames, thereby preventing repeated counting of the same pothole or roadside object. As a result, the generated object counts remain stable even when temporary detection failures occur, improving the overall reliability of the road damage analysis

5.5 Road Surface Masking

The Road Surface Masking module restricts road damage detections to the segmented road surface of each video frame. This significantly reduces false positive detections on surrounding objects such as vegetation, sidewalks, buildings, vehicles, and roadside infrastructure, thereby improving the reliability of road damage analysis.

Road Surface Segmentation Using YOLO11 Instance Segmentation

The proposed system employs a dedicated **YOLO11 Instance Segmentation** model (`exp-2.pt`) trained specifically for road surface segmentation. The model was trained on a separately annotated dataset in which the drivable road surface was labelled as a single segmentation class. During inference, the model produces a pixel-level segmentation mask representing the road region for every input frame.

Compared to traditional colour-based or region-of-interest methods, the deep learning approach learns the visual characteristics of roads directly from annotated data, enabling robust segmentation under varying lighting conditions, road textures, shadows, and environmental backgrounds.

The road segmentation model is loaded together with the road damage detection model during system initialization:

```
self.road_mask_model = YOLO("exp-2.pt")
self.damage_model = YOLO("best.pt")
```

For every input frame, the road segmentation model is executed first to generate the road mask:

```
road_results = self.road_mask_model.predict(  
    frame,  
    imgsz=800,  
    conf=0.08,  
    verbose=False  
) [0]
```

The predicted segmentation mask is resized to the original frame resolution and converted into a binary road mask, where road pixels are assigned a value of 1 and all remaining pixels are assigned a value of 0.

Application to Road Damage Detection

The generated road mask is applied as a logical constraint to all road damage classes:

```
ROAD_ONLY_CLASSES = {  
    "alligator cracking",  
    "crack",  
    "pothole",  
    "ravelling",  
    "rutting"  
}
```

For every detected road damage mask, a logical AND operation is performed with the binary road mask. Any segmented region that does not overlap the detected road surface is discarded before further processing.

Classes such as **guard rails**, **line markings**, **traffic signs**, and **trees** are excluded from road mask filtering because these objects naturally occur outside or along the road boundary.

Applying the road mask before tracking and area estimation substantially reduces false detections on non-road regions, resulting in more accurate damage localization, object counting, and measurement of road defects.

6. Dashboard

The dashboard provides the user-facing interface of the Road Condition Detection system. It allows users with no programming knowledge to upload a road survey video, configure detection parameters, run the complete inference pipeline, and view and download structured results — all through a browser-based web application requiring no command-line interaction. This chapter describes the tool selection rationale, the complete feature set, the user workflow, and each component of the dashboard interface in detail.

6.1 Dashboard Features

The dashboard implements the following complete feature set:

Input and Configuration

- Video upload via drag-and-drop file uploader supporting MP4, AVI, MOV, and MKV formats
- Configurable model weights path for both the road mask model and damage detection model
- Adjustable inference image size selection between 640 and 800 pixels
- Adjustable frame skip interval from 1 to 5 via slider
- Adjustable confidence threshold from 0.10 to 0.80 via slider
- Configurable camera parameters — altitude, focal length, and sensor height — via numeric input fields in the sidebar

Processing

- Single-click inference pipeline execution covering detection, road masking, tracking, area measurement, length measurement, mask rendering, and video annotation
- Progress spinner displayed during model loading and inference
- Automatic H.264 re-encoding of annotated output video for browser compatibility

Results Display

- Five KPI metric cards showing unique pothole count, unique tree count, total pothole area in m², total ravelling area in m², and total crack length in metres
- Annotated output video playback directly in the browser
- Per-class area measurement table with total, average, maximum, and minimum values in both pixels and square metres

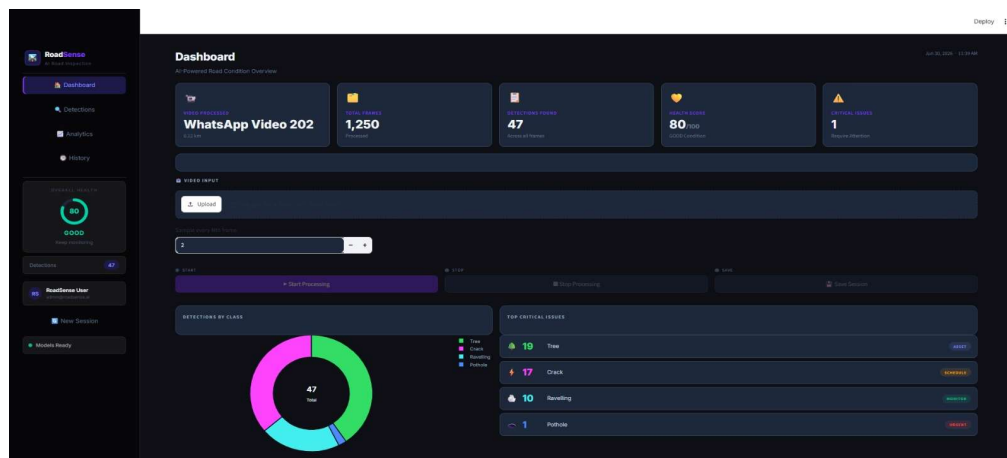
- Per-class crack length table with total and average values in pixels and metres
- Per-frame detection count line chart for all six damage and object classes
- Area calculation formula display with computed GSD value based on configured camera parameters

Export

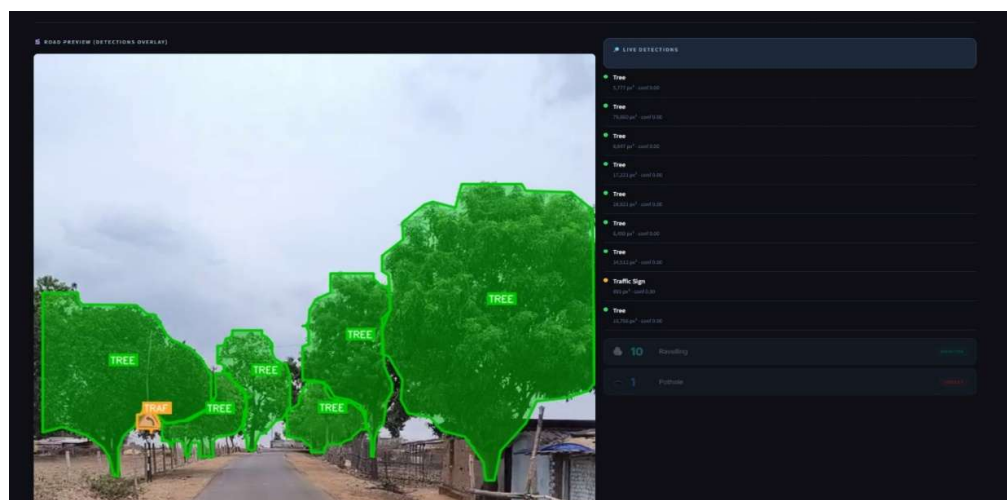
- Annotated video download as H.264 encoded MP4
- Per-frame statistics JSON report downloads

6.2 Dashboard Screenshots

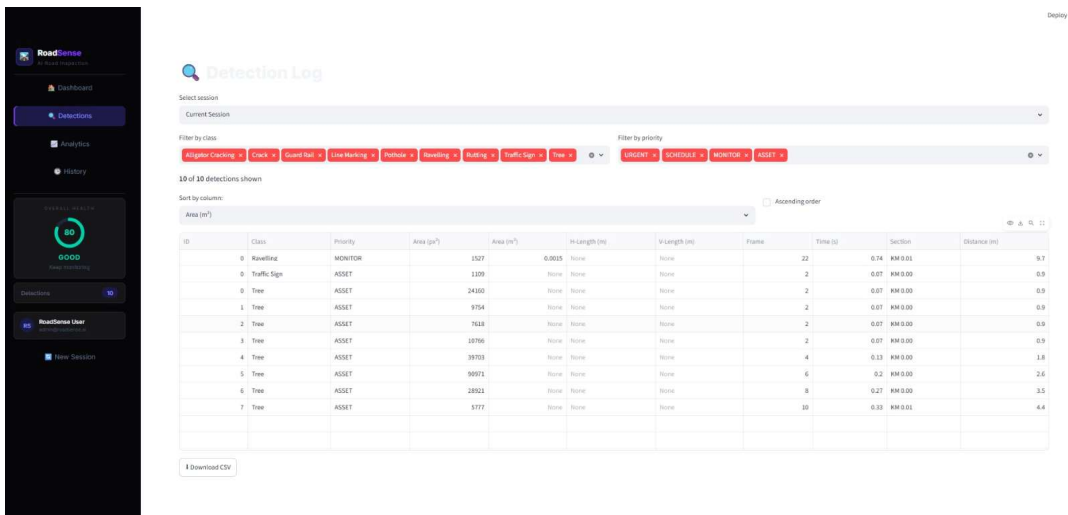
This section presents a complete visual walkthrough of the dashboard through annotated screenshots taken during a test run using a road survey video.



Screenshot 6.1- Dashboard main page



Screenshot 6.2- Dashboard live detection page



Screenshot 6.3- Dashboard detection page

7.Results and Evaluation

This chapter presents the complete results of the Road Condition Detection system across all model training experiments and the final deployed pipeline. Results are reported for each model variant trained during the project — YOLOv11s-seg, YOLOv11m-seg, YOLOv11l-seg, and YOLOv26l-seg — along with the road surface mask model. Evaluation metrics, confusion matrix analysis, PR curve interpretation, sample detection outputs, and measurement results from video inference are all presented here.

7.1 Evaluation Metrics

Model performance is evaluated using standard object detection and instance segmentation metrics. All metrics are computed on the held-out validation set, which the model never sees during training.

Metric	Abbreviation	Threshold	Relevance
Mean Average Precision	mAP50	$\text{IoU} \geq 0.50$	Primary metric
Strict Mean Average Precision	mAP50-95	$\text{IoU } 0.50\text{--}0.95$	Mask quality
Precision	P	—	False positive rate
Recall	R	—	Miss rate

Table 7.1 — Evaluation Metric Summary

7.2 Results: YOLOv11s-seg

YOLOv11s-seg was the first model trained as an initial experiment to verify dataset configuration and establish a performance baseline. Training was conducted on Kaggle using a single T4 GPU.

7.3 Results: YOLOv11m-seg

YOLOv11m-seg was trained as the second experiment with approximately 20.1 million parameters — more than double the capacity of YOLOv11s. Training was again conducted on Kaggle T4.

Observations

YOLOv11m showed meaningful and consistent improvement over YOLOv11s across most classes. The increased parameter count enabled the model to learn more complex and discriminative feature representations. High-frequency classes including tree, guard rails, and line marking showed the most significant gains. Damage classes such as pothole

and crack also improved noticeably. Rare classes including rutting continued to underperform due to the limited number of training samples, confirming that model capacity alone cannot compensate for data scarcity on severely underrepresented classes.

7.4 Results: YOLOv11l-seg (Ultralytics)

YOLOv11l-seg was the third model evaluated during this study, consisting of approximately 25.3 million parameters. The model was trained using the Ultralytics Hub platform by employing the full augmentation strategy and two-stage training methodology described in Chapter 5.

Metric	Value
Precision	63.1%
Recall	58.2%
mAP50	54.8%
mAP50-95	35.6%

Table 7.2 — YOLOv11l-seg Overall Results (Ultralytics Hub)



Screenshot 7.1-Performance Metrics of the Trained YOLOv11l-seg Model

7.5 Results: YOLOv26l-seg — Final Model (Ultralytics Hub)

The final and best-performing road damage detection model was YOLOv26l-seg trained on Ultralytics Hub using an NVIDIA RTX PRO 6000 GPU. This experiment, identified as exp-4 on the Ultralytics Hub platform, represents the culmination of the model progression and produced the results used in the deployed inference pipeline.

Metric	Value
mAP50 (M)	78.8%
mAP50-95 (M)	50.4%
Precision (M)	79.6%
Recall (M)	73.0%
Training GPU	NVIDIA RTX PRO 6000
Training duration	1 hour
Compute cost	\$2.11
Ultralytics version	8.4.67

Table 7.3- YOLOv26l-seg Overall Results (Ultralytics Hub)

7.6 Results: YOLOv11l-seg Road Mask Model

The road surface mask model was trained separately using YOLOv11l-seg to perform binary segmentation of the drivable road surface. This model operates as the first stage of the inference pipeline, generating the road boundary mask that constrains damage detections.

Metric	Value
Precision	90.2%
Recall	81.5%
mAP50	84.6%
mAP50-95	66.2%
Classes	Road surface (1)
Training platform	Ultralytics Hub

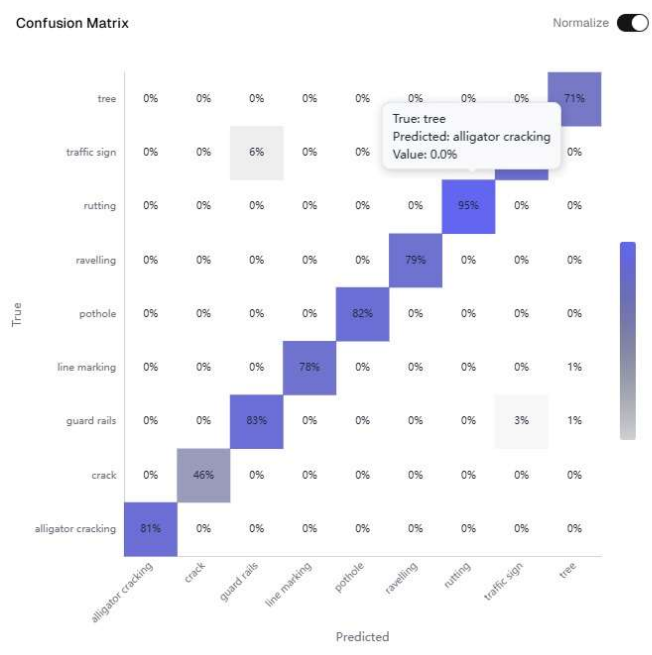
Table 7.4- YOLOv11l-seg Road Mask Model Results



Screenshot 7.2-Performance Metrics of Road Mask Model

7.7 Confusion Matrix

The confusion matrix provides a visual summary of classification performance, showing for each true class how often the model predicted each possible class. It reveals systematic misclassification patterns that aggregate metrics such as mAP do not expose.



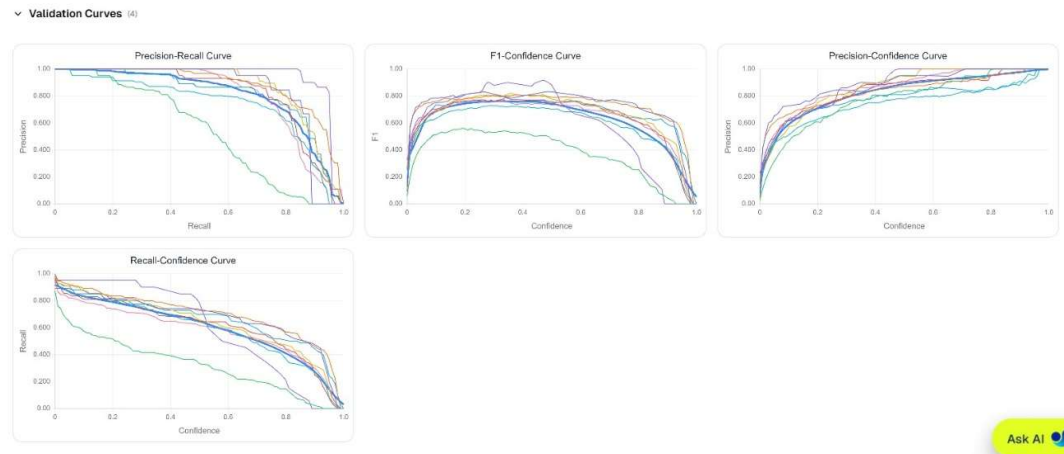
Screenshot 7.3- Confusion Matrix of YOLOv26l-seg model

Interpretation

In a well-performing model, the confusion matrix should show high values along the diagonal — indicating that each class is predominantly predicted as itself — and low values off the diagonal — indicating rare misclassification between classes.

7.8 PR Curve Analysis

The Precision-Recall curve plots precision against recall at varying confidence thresholds for each class. It provides a complete picture of the precision-recall trade-off — at high confidence thresholds the model makes fewer but more reliable detections (high precision, low recall), while at low thresholds it detects more objects including more false positives (low precision, high recall).



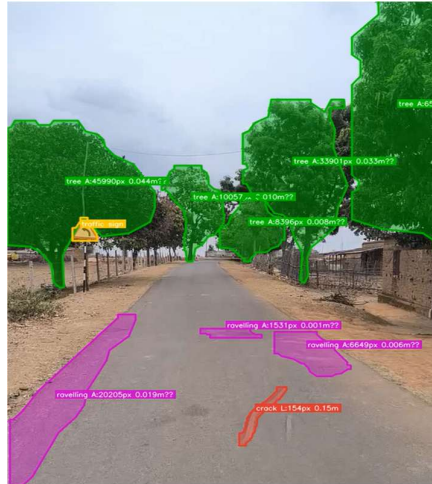
Screenshot 7.4- PR Curve of YOLOv26l-seg model

Interpretation

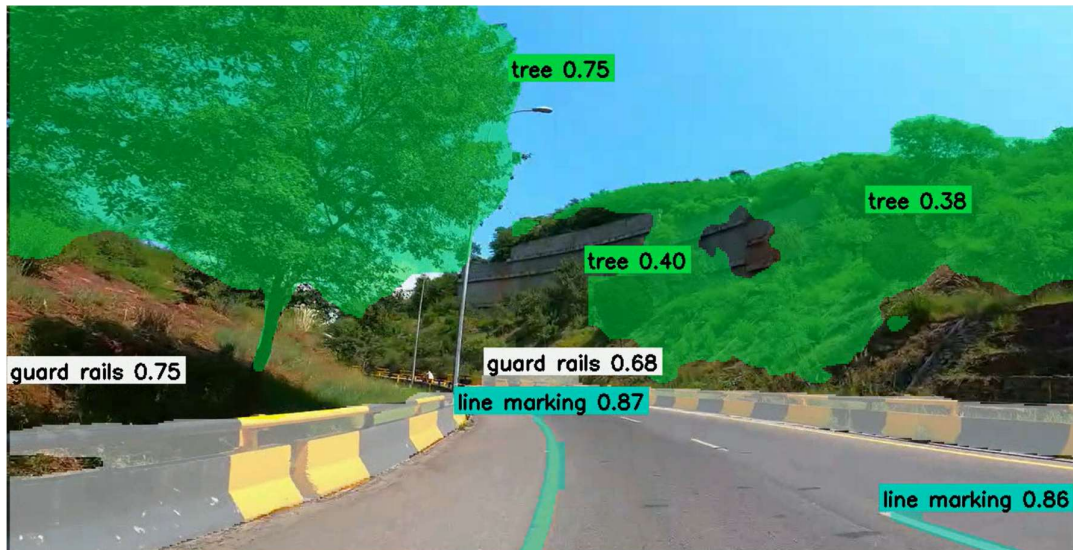
The area under the PR curve for each class equals the Average Precision for that class at IoU=0.50. Classes with curves that remain high across the full recall range — staying close to the top-right corner of the plot — indicate consistently reliable detection. Classes whose curves drop steeply at low recall values indicate that the model achieves good precision only on the most obvious instances of that class.

7.9 Sample Detection Outputs

This section presents sample frames from the annotated output video demonstrating the system's detection and segmentation performance on road survey footage under different conditions.



Screenshot 7.5- Output frame showing multiple damage classes detected simultaneously



Screenshot 7.6- frame showing tree and guard rail detection on a road with roadside infrastructure, demonstrating correct detection of roadside object classes

7.10 Area and Length Measurement Results

This section presents the area and length measurement results obtained from running the complete inference pipeline on the test road survey video.

Class	Total Detections	Total Area (px)	Total Area (m ²)	Avg Area (m ²)	Max Area (m ²)
Pothole	26	116828	2.9207	0.0199	0.0589
Ravelling	449	1023567	20.2824	0.0452	0.1615

Table 7.5- Area Measurement Results from Test Video

Class	Total Detections	Total Length (px)	Total Length (m)	Avg Length (m)
Crack	19	10738	19.9494	1.05

Table 7.6- Crack Length Measurement Results from Test Video

Class	Unique Count
Pothole	26
Tree	39

Table 7.7- Unique Object Count Results from Test Video

Camera Parameters Used for Measurement

The following camera parameters were used to compute GSD for all measurements in this section:

Parameter	Value
Camera altitude (H)	1.5 m
Sensor height (S_h)	3.6 mm
Focal length (f)	4.3 mm
Computed GSD	~0.001962 m/pixel


```
{} detection_report.json X
C: > Users > ANSHIKA > Downloads > {} detection_report.json > {} length_summary > {} crack > {} avg_m
1
2 {
3   "output_video": "annotated.mp4",
4   "total_frames": 1250,
5   "video_duration_s": 41.67,
6   "unique_counts": {
7     "pothole": 11,
8     "traffic sign": 0,
9     "tree": 17,
10    "ravelling": 15,
11    "crack": 19,
12    "guard rails": 0,
13    "line marking": 0,
14    "alligator cracking": 1,
15    "rutting": 0
16  },
17  "avg_per_frame": {
18    "crack": 1.41,
19    "tree": 1.81,
20    "pothole": 1.3,
21    "ravelling": 1.09,
22    "alligator cracking": 1.0
23  },
24  "max_per_frame": {
25    "crack": 4,
26    "tree": 4,
27    "pothole": 4,
28    "ravelling": 3,
29    "alligator cracking": 1
30  },
31  "area_summary": {
32    "tree": {
33      "count": 17,
34      "total_m2": 4.9462,
35      "avg_m2": 0.291,
36      "max_m2": 0.719
37    },
38    "ravelling": {
```

Screenshot 7.7- JSON file showing information summary

Deploy

ID	Class	Priority	Area (px ²)	Area (m ²)	H-Length (m)	V-Length (m)	Frame	Time (s)	Section	Distance (m)
6	Ravelling	MONITOR	12689	0.0489	None	None	198	6.6	KM 0.09	87.1
9	Ravelling	MONITOR	11736	0.0452	None	None	250	8.33	KM 0.11	110
8	Ravelling	MONITOR	11667	0.0449	None	None	244	8.13	KM 0.11	107.4
9	Crack	SCHEDULE	10738	0.0413	1.089	0.3866	118	3.93	KM 0.05	51.9
7	Ravelling	MONITOR	10651	0.041	None	None	240	8	KM 0.11	105.6
6	Crack	SCHEDULE	9999	0.0385	1.0459	0.3866	82	2.73	KM 0.04	36.1
4	Crack	SCHEDULE	9321	0.0359	0.777	0.465	50	1.67	KM 0.02	22
2	Ravelling	MONITOR	7209	0.0278	None	None	166	5.53	KM 0.07	73
8	Crack	SCHEDULE	6912	0.0266	0.7103	0.365	110	3.67	KM 0.05	48.4
0	Pothole	URGENT	6477	0.0249	None	None	146	4.87	KM 0.07	64.2
10	Crack	SCHEDULE	5535	0.0213	0.6201	0.053	204	6.8	KM 0.09	89.8
3	Crack	SCHEDULE	5532	0.0213	0.4336	0.4101	34	1.13	KM 0.02	15

Download CSV

Screenshot 7.8- Dashboard showing information summary

8. Conclusion and Future Work

This chapter summarises the work completed throughout the Road Condition Detection project, presents the key findings drawn from the experimental results, discusses the practical implications of the system for road survey automation, and outlines directions for future enhancement that would extend the system's capabilities and address the limitations identified in Chapter 9.

8.1 Summary of Work Done

This project aimed to develop an automated road condition detection system capable of identifying, segmenting, tracking, counting, and measuring road defects and roadside objects from vehicle-mounted road survey videos, and presenting the results through an interactive web-based dashboard. The complete system was designed, implemented, trained, and evaluated during a two-month internship at Bhaskaracharya National Institute for Space Applications and Geo-informatics (BISAG-N), Gandhinagar, by two B.Tech students from Gati Shakti Vishwavidyalaya.

Dataset Preparation

A custom dataset comprising **2,252 annotated images** was prepared using two complementary data sources. The primary dataset was assembled from publicly available road damage datasets obtained from Kaggle and Roboflow, while additional road survey videos collected by BISAG-N were used to improve representation of real-world Indian road conditions. Frames extracted from the BISAG-N videos were manually annotated using Roboflow with polygon-level instance segmentation masks. The final dataset consisted of **2,847 annotated segmentation masks** covering road damage classes and roadside objects. The dataset was divided into training, validation, and testing subsets and exported in YOLO instance segmentation format.

Model Training

A systematic progression through multiple model variants was performed for road damage detection. Training began with YOLO11s-seg as a baseline model, followed by YOLO11m-seg and YOLO11l-seg to improve segmentation accuracy. The final road damage detection model was trained using **YOLO26l-seg** on the Ultralytics platform. A separate **YOLO11l-seg** road segmentation model (exp-2.pt) was independently trained to segment the drivable road surface, providing a spatial constraint for subsequent damage detection.

Inference Pipeline

A complete two-stage inference pipeline was implemented in Python. For every input frame, the road segmentation model first generates a binary road mask, after which the

road damage detection model performs instance segmentation of road defects and roadside objects. The predicted damage masks are filtered using the road mask through a logical AND operation, followed by overlap handling, small-object filtering, and visualization. Ground Sampling Distance (GSD) is used to estimate the real-world area of segmented road defects. Multi-object tracking is performed using the **ByteTrack** algorithm to maintain persistent object identities and compute unique counts of potholes and trees throughout the video.

Dashboard

An interactive Streamlit dashboard was developed as the user interface for the complete system. The dashboard accepts road survey videos, executes the full inference pipeline, and presents annotated video output together with summary statistics, object counts, defect area measurements, configurable camera parameters, and downloadable processed results. The dashboard provides an intuitive platform for visualizing road condition information and supports efficient analysis of road survey data for maintenance planning and infrastructure assessment.

8.2 Key Findings

The following key findings emerged from the experimental work conducted in this project.

Model Capacity Scales with Dataset Complexity

Across all four damage detection model experiments, a consistent pattern was observed — increasing model capacity from YOLOv11s through YOLOv11m, YOLOv11l, and YOLOv26l produced consistent improvement in overall mAP50. For a nine-class segmentation task with a small imbalanced dataset, model capacity is a critical factor. The small YOLOv11s model with 9.4 million parameters was demonstrably insufficient for this task, while the large YOLOv26l model provided the best generalisation.

GPU Quality Dominates Training Efficiency

The progression from Kaggle T4 free GPU training — where a 300-epoch YOLOv11l run required multiple 12-hour sessions — to Ultralytics Hub RTX PRO 6000 training — where YOLOv26l completed in one hour — demonstrated that GPU quality has a far more dominant effect on training efficiency than hyperparameter tuning or architecture selection alone. For student projects working with large segmentation models, access to professional-grade GPU infrastructure, even for a brief paid session, produces significantly better practical outcomes than extended free-tier training.

Class Imbalance is the Primary Performance Bottleneck

The most significant factor limiting the final model's mAP50 of 54.8% was not model architecture or training duration but rather the extreme class imbalance in the training data.

Classes with sufficient representation — tree with 1,680 instances, ravelling with 827, crack with 791 — achieved strong detection performance. Classes with limited representation — rutting with 58 instances, alligator cracking with 130 — consistently underperformed across all model variants regardless of augmentation strategy. This confirms that for real-world road damage detection, balanced and representative data collection is more valuable than architectural improvements.

Two-Model Pipeline Improves Detection Specificity

The use of a dedicated trained road mask model as a spatial filter for damage detections — rather than relying solely on the damage detection model to avoid non-road detections — measurably reduced false positive detections on non-road surfaces. The trained road mask model provided more accurate and context-aware road boundary delineation compared to colour-based fallback approaches, particularly on roads with unusual surface colours, heavy shadow, or bright markings.

Augmentation is Essential for Small Datasets

The combination of mosaic, copy-paste, multi-scale, and photometric augmentation applied during training was essential for achieving reasonable generalisation on a 1,300-image dataset. Without these augmentation strategies, the model would have overfit rapidly to the training set and failed to generalise to the visual diversity present in real Indian road survey footage. Copy-paste augmentation in particular provided measurable improvement in rare class detection by artificially increasing the frequency of underrepresented class appearances in training batches.

Pixel-to-Metric Conversion is Feasible Without Specialised Hardware

The GSD-based area and length measurement approach demonstrated that physically meaningful real-world measurements can be extracted from standard monocular road survey video without requiring specialised depth sensors, stereo cameras, or GPS-IMU integration. While the absolute accuracy of measurements depends on the accuracy of camera parameter inputs, the methodology is sound and produces estimates that are useful for comparative damage assessment and maintenance prioritisation.

8.3 Future Enhancements

Several directions for future development would substantially extend the system's capabilities and address the limitations documented in Chapter 9.

Expanded and Balanced Dataset

The most impactful single improvement would be expanding the training dataset with a focus on underrepresented classes. Collecting and annotating several hundred additional images specifically of rutting and alligator cracking — from a variety of road types, lighting conditions, and damage severities — would likely produce a larger improvement in overall mAP50 than any architectural or training strategy change. A target of 500 or

more annotated instances per class would provide sufficient representation for reliable detection across all nine classes.

GPS Integration for Spatial Damage Mapping

Integrating GPS data from the survey vehicle with the inference pipeline would allow detected damage instances to be georeferenced — assigned real-world coordinates corresponding to their location on the road network. This would enable the generation of spatially referenced road damage maps that maintenance authorities could use to plan repair operations, prioritise road segments, and track damage progression over time through repeated surveys. The GPS track could be synchronised with video timestamps to associate each detected damage instance with its geographic coordinates.

Pavement Condition Index Scoring

The current system reports raw counts and measurements but does not produce a composite road condition score. Implementing a weighted scoring formula — for example assigning higher weights to structurally severe damage types such as potholes and alligator cracking, and lower weights to surface-level damage such as ravelling — would produce a single road condition index value per road segment. This index could be mapped along the road network to provide an immediate visual summary of road condition suitable for presentation to non-technical stakeholders such as municipal engineers and government officials.

Association Rule Mining

The detection outputs contain co-occurrence information — which damage types appear together in the same frames — that can be analysed using association rule mining to discover statistically significant damage co-occurrence patterns. For example, a rule such as alligator cracking → pothole with 78% confidence would indicate that road segments showing alligator cracking are highly likely to develop potholes if not addressed. These rules could inform preventive maintenance strategies by flagging early-stage damage patterns associated with future severe damage.

Real-Time Edge Deployment

The current system operates as an offline batch processor — survey videos are recorded, transferred, and processed after the fact. Deploying the inference pipeline on an embedded GPU module — such as the NVIDIA Jetson series — mounted directly in the survey vehicle would enable real-time processing during the survey itself. This would allow immediate alerts for severe damage detection, on-the-fly GPS tagging of detected damage, and elimination of the post-processing step, reducing the time from survey to actionable results from days to seconds.

Per-Class Confidence Threshold Optimisation

The current pipeline applies a single global confidence threshold of 0.30 across all classes. Since different classes have different confidence distributions — the model is inherently more confident about tree and guard rail detections than about rutting and alligator cracking — a class-specific threshold would improve the precision-recall balance for each class independently. Optimal per-class thresholds could be determined by analysing the PR curve for each class on the validation set and selecting the threshold that maximises the F1 score for that class.

Severity Classification

The current system detects and segments damage instances but does not classify their severity. Extending the annotation scheme to include severity labels — for example classifying potholes as minor, moderate, or severe based on depth and diameter — would allow the system to prioritise repair urgency. Severity classification could be implemented either as additional output classes in the segmentation model or as a secondary classification network applied to each detected instance mask.

Depth Estimation Integration

Integrating a monocular depth estimation model — such as MiDaS or Depth Anything — alongside the damage detection pipeline would provide per-pixel depth estimates that could significantly improve area and volume measurement accuracy. Depth information would allow the system to account for road surface slope and camera tilt, correct for perspective distortion across the frame, and estimate the volume of pothole depressions rather than just their surface area — a more physically meaningful measure of structural damage severity.

3D Reconstruction for Volume Measurement

For the most accurate damage quantification, structure-from-motion or stereo camera based 3D reconstruction of the road surface would allow volumetric measurement of potholes and depression depth of rutting channels. While this represents a significant hardware and computational investment, it would transform the system from a detection and classification tool into a full quantitative road condition assessment platform capable of supporting engineering-grade maintenance decisions.

8.4 Real-World Deployment Considerations

Deploying the Road Condition Detection system in a production road survey environment raises several practical considerations beyond the technical scope of this project.

Camera Standardisation

For area and length measurements to be comparable across different survey runs, the camera hardware and mounting position must be standardised across all survey vehicles. Variation in camera altitude, focal length, or tilt between vehicles would introduce systematic measurement biases that make cross-survey comparisons unreliable. A calibration protocol — measuring and verifying camera parameters before each survey season — should be established as part of the operational procedure.

Lighting and Weather Conditions

The model was trained primarily on daytime footage under normal weather conditions. Performance under rain, fog, night conditions with artificial lighting, or direct low-angle sunlight has not been evaluated and may be significantly degraded. Extending the training dataset to include footage under diverse weather and lighting conditions would be necessary before operational deployment across all survey conditions encountered on Indian roads through different seasons.

Operator Training

The Streamlit dashboard is designed for ease of use but requires operators to understand the significance of confidence thresholds and camera parameters to interpret results correctly. Operator training covering the meaning of detection metrics, the limitations of the area measurement methodology, and the expected failure modes of the system — particularly for rare damage classes — would be necessary for responsible operational use.

Periodic Model Retraining

Road surface materials, road construction standards, and the types of damage prevalent on Indian roads may evolve over time. The model trained on the current dataset reflects the visual characteristics of road damage as captured in the 2025–2026 survey footage. Periodic retraining on newly collected survey data would maintain detection performance as road characteristics change and as the survey vehicle camera hardware is upgraded.

Integration with Existing Road Management Systems

For maximum operational value, the detection outputs should be integrated with existing road asset management systems used by road authorities and municipal bodies. This would allow detected damage records to automatically populate maintenance work order systems, update road condition databases, and feed into budget planning workflows without requiring manual data transcription from the dashboard outputs.

9. References

[1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

<https://arxiv.org/abs/1506.02640>

[2] K. He, G. Gkioxari, P. Dollár and R. Girshick, "Mask R-CNN," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

<https://arxiv.org/abs/1703.06870>

[3] A. Kirillov et al., "Segment Anything," *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

<https://arxiv.org/abs/2304.02643>

[4] G. Jocher et al., "Ultralytics YOLO11 Documentation," Ultralytics, 2024.

<https://docs.ultralytics.com>

[5] Ultralytics, "YOLO26 Documentation," 2025.

<https://docs.ultralytics.com>

[6] G. Jocher et al., "Ultralytics YOLOv8," GitHub Repository, 2023.

<https://github.com/ultralytics/ultralytics>

[7] Y. Zhang, L. Yang, Y. Zhang and Y. Zhu, "Road Crack Detection Using Deep Convolutional Neural Network," *IEEE International Conference on Image Processing (ICIP)*, 2016.

<https://ieeexplore.ieee.org/document/7533052>

[8] D. Arya et al., "RDD2022: A Multi-national Image Dataset for Automatic Road Damage Detection," arXiv, 2022.

<https://arxiv.org/abs/2209.08538>

[9] A. Maeda et al., "Road Damage Detection Using Deep Neural Networks with Images Captured Through a Smartphone," *arXiv*, 2018.

<https://arxiv.org/abs/1801.09454>

[10] A. Bochkovskiy, C. Wang and H. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv*, 2020.

<https://arxiv.org/abs/2004.10934>

[11] C. Wang, A. Bochkovskiy and H. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," *CVPR*, 2023.

<https://arxiv.org/abs/2207.02696>

[12] Y. Ge, Z. Qin, S. Li et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," *ECCV*, 2022.

<https://arxiv.org/abs/2110.06864>

[13] Roboflow, "Roboflow Annotation Platform."

<https://roboflow.com>

[14] OpenCV Team, "OpenCV Documentation."

<https://docs.opencv.org>

[15] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

<https://opencv.org>

[16] Streamlit Inc., "Streamlit Documentation."

<https://docs.streamlit.io>

[17] NumPy Developers, "NumPy Documentation."

<https://numpy.org/doc>

[18] SciPy Developers, "SciPy Documentation."

<https://scipy.org>

[19] PyTorch Team, "PyTorch Documentation."

<https://pytorch.org/docs>

[20] Kaggle, "Kaggle Notebooks."

<https://www.kaggle.com>

[21] Ultralytics, "Ultralytics HUB."

<https://hub.ultralytics.com>

[22] ASTM International, "ASTM D6433 – Standard Practice for Roads and Parking Lots Pavement Condition Index Surveys," 2018.

<https://www.astm.org/d6433>

[23] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd Edition, Springer, 2022.

<https://szeliski.org/Book>

[24] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Cambridge University Press, 2004.

<https://www.robots.ox.ac.uk/~vgg/hzbook/>

[25] P. Corke, *Robotics, Vision and Control*, Springer, 2017.

<https://petercorke.com>

[26] R. Gonzalez and R. Woods, *Digital Image Processing*, 4th Edition, Pearson, 2018.

[27] R. Jain, R. Kasturi and B. Schunck, *Machine Vision*, McGraw-Hill, 1995.

[28] C. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[29] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

<https://www.deeplearningbook.org>

[30] NVIDIA Corporation, "CUDA Toolkit Documentation."

<https://docs.nvidia.com/cuda>

[31] FFmpeg Developers, "FFmpeg Documentation."

<https://ffmpeg.org/documentation.html>

[32] Python Software Foundation, "Python 3 Documentation."

<https://docs.python.org/3>

[33] Google Colaboratory Documentation.

<https://colab.research.google.com>

[34] Scikit-image Developers, "Scikit-image Documentation."

<https://scikit-image.org>

[35] OpenStreetMap Foundation.

<https://www.openstreetmap.org>

10. Appendix

Appendix A - Dataset Configuration (data.yaml)

```
train: ../train/images
val: ../valid/images
test: ../test/images

nc: 9
names: ['alligator cracking', 'crack', 'guard rails', 'line
marking', 'pothole', 'ravelling', 'rutting', 'traffic sign',
'tree']

roboflow:
  workspace: anshikas-workspace-bqhd0
  project: damages_of_road
  version: 2
  license: CC BY 4.0
  url: https://universe.roboflow.com/anshikas-workspace-bqhd0/d
```

Appendix B - requirements.txt

```
streamlit>=1.35.0
opencv-python>=4.9.0
ultralytics>=8.2.0
numpy>=1.26.0
pillow>=10.3.0
pandas>=2.2.0
plotly>=5.22.0
folium>=0.16.0
streamlit-folium>=0.20.0
fpdf2>=2.7.9
supervision>=0.21.0
torch>=2.3.0
torchvision>=0.18.0
scipy>=1.13.0
scikit-image>=0.22.0
networkx>=3.2
```

C - Project Folder Structure

```
RoadSense/  
├── models/  
│   ├── best.pt  
│   └── exp-2.pt  
├── dashboard/  
│   ├── app.py  
│   └── utils.py  
├── outputs/  
├── videos/  
├── dataset/  
├── requirements.txt  
└── README.md
```

Appendix D - Dashboard Output JSON

```
{  
  "output_video": "annotated.mp4",  
  "total_frames": 1250,  
  "video_duration_s": 41.67,  
  "unique_counts": {  
    "pothole": 11,  
    "traffic sign": 0,  
    "tree": 17,  
    "ravelling": 15,  
    "crack": 19,  
    "guard rails": 0,  
    "line marking": 0,  
    "alligator cracking": 1,  
    "rutting": 0  
  },  
  "avg_per_frame": {  
    "crack": 1.41,  
    "tree": 1.81,  
    "pothole": 1.3,  
    "ravelling": 1.09,  
  }  
}
```

```

    "alligator cracking": 1.0
  },
  "max_per_frame": {
    "crack": 4,
    "tree": 4,
    "pothole": 4,
    "ravelling": 3,
    "alligator cracking": 1
  },
  "area_summary": {
    "tree": {
      "count": 17,
      "total_m2": 4.9462,
      "avg_m2": 0.291,
      "max_m2": 0.719
    },
    "ravelling": {
      "count": 15,
      "total_m2": 1.1659,
      "avg_m2": 0.0777,
      "max_m2": 0.171
    },
    "pothole": {
      "count": 11,
      "total_m2": 0.3256,
      "avg_m2": 0.0296,
      "max_m2": 0.0754
    },
    "alligator cracking": {
      "count": 1,
      "total_m2": 0.051,
      "avg_m2": 0.051,
      "max_m2": 0.051
    }
  }
}

```

Appendix E - Source Code Snippets

Road Health Score Computation

```
def compute_health_score(class_counts: dict,
                        detections: list = None,
                        total_frames: int = 0) -> int:

    total_damage_m2 = sum(
        d.get("area_m2") or 0
        for d in detections
        if d.get("class") in ROAD_DAMAGE_CLASSES
    )

    total_road_m2 = total_frames * DEFAULT_FRAME_AREA_M2
    coverage = total_damage_m2 / total_road_m2

    if coverage >= COVERAGE_CRITICAL:
        score = _lerp(coverage, COVERAGE_CRITICAL,
                     COVERAGE_CRITICAL * 2, 39, 0)

    return max(0, min(100, score))
```

Detection Logging and Filtering

```
mask = (
    df["class"].isin(sel_keys)
    & df["Priority"].isin(pri_sel)
    & (df["confidence"] >= conf_min)
)

filtered = df[mask]
```



Report Verification Procedure

Date:

Project Name: Road Condition Detection

Student Name & ID: 1. Omana Arya (26BISAG0495)

2. Anshika Tiwari(26BISAG0496)

	Soft Copy	Hard Copy
Report Format:		
Project Index:		

Sign by Training Coordinator

Sign by Project Guide